GOA UNIVERSITY
Goa Business School
Taleigao Plateau, Goa

PROJECT REPORT

LIBGRAM

LIBRARY MANAGEMENT SYSTEM

A Java 21 Desktop Application using Swing and CSV-Based Persistence
with its companion personal edition, LibGramPocket

Course: Object Oriented Technology Lab (OOTL)
Programme: Master of Computer Applications (MCA)
Mini-Project Report

Submitted by
Yeshwant Shetgaonkar
Roll No. 2661

Yash Gaonkar
Roll No. 2660

Under the guidance of
Sir Hanumant Redkar

Submitted to
Goa Business School
Goa University, Taleigao, Goa

Date of submission: 28 October 2026

Certificate

This is to certify that the project report entitled **"LibGram: Library Management System"** is submitted by **Yeshwant Shetgaonkar (Roll No. 2661)** and **Yash Gaonkar (Roll No. 2660)** of the Master of Computer Applications (MCA) programme, Goa Business School, Goa University, Taleigao, Goa, in partial fulfilment of the academic requirements of the course **Object Oriented Technology Lab**.

The report describes the field inquiry, analysis, design, implementation and testing of a Java desktop application and its companion personal edition. To the best of our knowledge, the work was carried out by the students under guidance and has not been submitted elsewhere for any other purpose.

Role	Name / Signature	Date
Project Guide	Sir Hanumant Redkar	
Program Director		
Examiner		

Institutional seal:

Declaration

We hereby declare that the project report entitled **"LibGram: Library Management System"** is a record of the work carried out by us for the Object Oriented Technology Lab of the MCA programme at Goa Business School, Goa University. The software, the diagrams and the report were prepared by us. Where the work of others has been consulted, it is acknowledged in the references.

The demonstration data used in the project is fictional. No personal data of real library members has been included in the software, the tests or this report.

Yeshwant Shetgaonkar (Roll No. 2661) **Yash Gaonkar** (Roll No. 2660)

Date: 28 October 2026

Acknowledgement

We express our sincere gratitude to our project guide, **Sir Hanumant Redkar**, for his guidance, encouragement and constructive suggestions throughout the development of this project and the preparation of this report.

We thank **Goa Business School, Goa University**, for providing the academic environment, the laboratory facilities and the opportunity to carry out this mini-project as part of the Object Oriented Technology Lab.

We are grateful to the staff of the rural panchayat libraries in Goa who took the time to speak with us about how they keep their records. Their experience is the reason this project exists, and we hope that the software is of use to them.

We also thank our classmates, friends and families for their support, patience and feedback during the project.

Yeshwant Shetgaonkar
Yash Gaonkar

Abstract

LibGram is a desktop Library Management System implemented using Java 21 and Swing. It supports book and member catalogues, lending and returns, reservations, charges, payments, notifications and settings.

The project began with a real incident: a member was told that a book had not been returned, although it had been, and the library's old paper register had no record of the return. A basic inquiry at rural panchayat libraries in Goa found the same practice of paper registers, and a comparison of existing library software showed that the available systems were too costly, too complex, or too heavy for the low-end computers in such libraries. LibGram was designed to fill that gap: it is a compact application, about 376 KB, that has been tested for libraries of up to 5,000 books and 500 members.

The project uses a layered design: Swing UI components call services; services implement business rules; repository interfaces isolate data access; CSV-backed repositories persist records; and model classes represent domain entities. The application includes documented safeguards such as atomic file replacement, backups and recovery, single-instance locking, logging, salted password hashes and a plain-JDK regression suite. A companion application, LibGramPocket, applies the same architecture to individuals who lend and borrow books.

This report covers the motivation, field inquiry, comparison of existing systems, requirements, brainstorming, accepted and rejected ideas, design diagrams, implementation, Object-Oriented concepts applied, testing, limitations and future enhancements.

Keywords: library management, Java, Swing, CSV persistence, rural libraries, Object-Oriented design, Goa

Table of Contents

1. Introduction 1
1.1 Purpose of this Report 1
1.2 Summary of the Solution 1
1.3 Organisation of the Report 2
2. Background and Motivation 3
2.1 Why a Library Management System? 3
2.2 The Incident that Started the Project 3
2.3 Field Inquiry 3
2.4 Why Existing Software Was Not the Answer 4
2.5 Why Java, Swing and CSV? 4
2.6 Motivation Summary 4
3. Problem Statement 5
3.1 Problem Decomposition 5
3.2 Problem Statement for LibGramPocket 5
4. Objectives 6
4.1 Primary Objective 6
4.2 Specific Objectives 6
4.3 Measurable Success Criteria 6
5. Scope of the Project 8
5.1 In Scope 8
5.2 Out of Scope 8
5.3 Assumptions 8
6. Existing System Analysis 9
6.1 The Manual Register System 9
6.2 Comparison of Existing Software 9
6.3 Gap Analysis 10
7. Proposed System 11
7.1 Overview 11
7.2 Main Capabilities 11
7.3 LibGramPocket 11
7.4 Design Principles 12
8. Stakeholders and Roles 13
8.1 Stakeholders 13
8.2 Roles and Permissions in LibGram 13
9. Requirements 14
9.1 Functional Requirements 14
9.2 Non-Functional Requirements 15
9.3 LibGramPocket Requirements 15
10. Feasibility Study 16
10.1 Technical Feasibility 16
10.2 Economic Feasibility 16
10.3 Operational Feasibility 16
10.4 Schedule Feasibility 16
10.5 Legal and Ethical Feasibility 16
11. Brainstorming and Idea Generation 17
11.1 Idea Themes 17
11.2 How the Ideas Were Evaluated 18

12. Accepted and Rejected Ideas 19
12.1 Accepted Ideas 19
12.2 Rejected Ideas 19
12.3 Decision Rationale 19
13. Development Methodology 21
13.1 Stages 21
13.2 Phases of LibGram 21
13.3 Phases of LibGramPocket 22
13.4 Tools and Environment 22
14. System Architecture 23
14.1 Layered Architecture 23
14.2 Size of the Code Base 24
14.3 Start-up Sequence 24
14.4 Event-Driven Collaboration 24
15. System Design Diagrams 26
15.1 Use-Case Diagram 26
15.2 Activity Diagram: Lending 27
15.3 Sequence Diagram: Lending 28
15.4 Data-Flow Diagram 29
15.5 Conceptual Class Diagram 30
15.6 Design Decisions Visible in the Diagrams 30
16. Data Model and File Inventory 31
16.1 Domain Entities 31
16.2 Enumerations 32
16.3 Storage Layout 33
16.4 Source Tree 33
17. Implementation: Book Catalogue and Member Management 34
17.1 Book Catalogue Module 34
17.2 Member Management Module 35
18. Implementation: Lending, Returns and Reservations 37
18.1 Lending Module 37
18.2 Return Module 37
18.3 Lost Books 38
18.4 Reservations 38
19. Implementation: Charges, Billing and Payments 40
19.1 Design of the Billing Rules 40
19.2 Return Charges 40
19.3 Worked Examples 40
19.4 Charge Lifecycle 41
19.5 Payments and Dues 41
19.6 Fee Schedule Defaults 41
20. Implementation: Notifications and Alerts 43
20.1 Notification Types 43
20.2 The Alert Scanner 43
20.3 Using Notifications 43
21. Implementation: Authentication, Authorization and Settings 45
21.1 Authentication 45
21.2 Authorization 45
21.3 Settings and Configuration 45

22. Implementation: Persistence, Reliability and User Interface 47
22.1 CSV Persistence 47
22.2 Atomic Writes 47
22.3 Backups and Recovery 47
22.4 Single-Instance Lock 48
22.5 Logging and Error Handling 48
22.6 User Interface Design 48
23. LibGramPocket: The Personal Edition 50
23.1 Why a Second Application? 50
23.2 Features 50
23.3 Menu Structure 50
23.4 Data Model 50
23.5 Account Isolation and Storage 51
23.6 Behaviour Notes 51
23.7 Differences from LibGram 51
24. Object-Oriented Concepts Applied 53
24.1 Mapping of Concepts to the Project 53
24.2 Design Patterns Used 54
24.3 Why These Choices Matter 54
25. Build, Run and Deployment 55
25.1 Requirements 55
25.2 Building LibGram 55
25.3 Running 55
25.4 Demo Data 55
25.5 Building LibGramPocket 56
25.6 Deployment to a Library Computer 56
26. Testing 57
26.1 Testing Strategy 57
26.2 Test Inventory of LibGram 57
26.3 Test Inventory of LibGramPocket 58
26.4 Designed Test Cases 59
26.5 Test Execution and Results 61
27. Results, Limitations and Future Enhancements 63
27.1 Results Against Objectives 63
27.2 Limitations 63
27.3 Future Enhancements 64
27.4 Privacy, Security and Ethics 64
27.5 Risk Analysis 64
27.6 Project Management 65
28. Conclusion 66
29. References 67
Appendix A. Glossary 68
Appendix B. Demonstration Plan 69
Appendix C. Command Reference 70
Appendix D. Final Submission Checklist 71

1. Introduction

Libraries are among the oldest community institutions, and the quality of their service depends on a simple promise: when a member borrows a book, the library remembers it, and when the member returns it, the library remembers that too. Large libraries keep this promise with integrated library systems. Many small, rural libraries still keep it with a pen and a bound register. **LibGram** is a desktop Library Management System designed to bring the same reliability to the second group.

LibGram is implemented in Java 21 with the Swing toolkit. It manages the book catalogue and the member register, records lending and returns, handles reservations, calculates charges, records payments, raises notifications, and lets an administrator configure the rules of the library through a settings screen. The application is deliberately small: the packaged executable is a single JAR file of roughly 376 KB, and it stores its data in plain, human-readable CSV text files so that no database server has to be installed or maintained.

A companion application, **LibGramPocket**, applies the same architecture to a different audience: individuals who lend books to friends and borrow books from them. LibGramPocket is a JAR of roughly 276 KB (without data). It lets a person keep a private catalogue of owned books, a list of contacts, and an exact record of what was lent, what was borrowed, when it is due, and what is owed. Both applications were developed as part of the Object Oriented Technology Lab (OOTL) mini-project of the Master of Computer Applications (MCA) programme at Goa Business School, Goa University.

1.1 Purpose of this Report

This report documents the complete life of the project: why the topic was chosen, what was observed in the field, which alternatives were considered and rejected, how the system was analysed, designed, implemented and tested, and what remains to be done. It is organised so that a reader who has never seen the software can understand the problem, the reasoning behind each decision, and the evidence that the solution works.

1.2 Summary of the Solution

Table 1: LibGram and LibGramPocket at a glance

Aspect	LibGram	LibGramPocket
Intended user	Small and rural libraries (staff and administrator)	Individuals who lend and borrow books
Technology	Java 21, Swing, CSV text storage	Java 21, Swing, CSV text storage
Packaged size	About 376 KB (JAR)	About 276 KB (JAR, without data)
Field-tested capacity	Up to 5,000 books and 500 members	Up to 5,000 books
Accounts	One shared library store; roles ADMIN and STAFF	One private data folder per signed-in account

Aspect	LibGram	LibGramPocket
Core workflows	Catalogue, members, lending, returns, reservations, charges, payments, notifications, settings	Owned books, contacts, lending, borrowing, returns, charges, notifications, settings
Reliability features	Atomic writes, rolling backups, recovery, single-instance lock, logging	Atomic writes, rolling backups, recovery, single-instance lock, logging
Security	Salted PBKDF2 password hashes, role-based permissions	Salted PBKDF2 password hashes, account isolation

1.3 Organisation of the Report

Chapters 2 to 6 explain the motivation, the problem and the existing situation. Chapters 7 to 13 cover the proposed system, requirements, feasibility, brainstorming, decisions and development approach. Chapters 14 to 17 present the architecture, the UML diagrams and the data model. Chapters 18 to 22 describe the individual modules and the LibGramPocket application, and show how Object-Oriented principles were applied. Chapters 23 to 27 cover testing, user walkthroughs, results, limitations, future work, project management, risks and ethics, followed by the conclusion, references and appendices.

2. Background and Motivation

2.1 Why a Library Management System?

Libraries manage books, members, loans, due dates, reservations, charges and payments. These records are closely related: a single lending transaction changes the availability of a book, the borrowing count of a member, the due-date calendar, and potentially the amount a member owes. When these records are kept on paper, each of those links has to be maintained by hand, and every link is a chance for an error to enter the system.

2.2 The Incident that Started the Project

The idea for LibGram did not begin in a classroom. It began at a library counter. One member of the project team was told that a book had not been returned, even though the team member had returned it only a couple of days earlier. When the team member asked to see the records, the librarian brought out an old, poorly maintained register. The return was not written in it.

There was no malice in the situation, and nobody could prove what had happened. The return might never have been entered, might have been entered on the wrong page, or might have been lost when pages were damaged. This is exactly the point: with a paper register there is no reliable way to tell. The person at the counter has only the register to rely on, and a borrower has only memory. A member can be wrongly accused, and a librarian can wrongly lose a book, with equal ease.

> **Key insight.** The problem was not a lack of effort on the part of the librarian. It was the absence of a trustworthy, searchable and tamper-evident record. That is a software problem, and it is one that a small, well-built application can solve.

2.3 Field Inquiry

After the incident the team decided to find out whether it was an isolated case. A basic inquiry was carried out at two to three rural panchayat libraries in Goa. At each library the staff described the same situation: records are kept in registers, and this is how things have been done for decades. None of the libraries visited used any library software.

The inquiry was intentionally small and informal. It was not a statistical survey, and this report does not claim that it represents every library in the state. What it did provide was enough evidence that the incident was not unusual and that the need for a simple digital record was real.

Table 2: Summary of field findings

Observation	Finding
Libraries visited	Two to three rural panchayat libraries in Goa
Method	Direct conversations with library staff and observation of the record-keeping practice
Current record system	Paper registers, in use "for decades"
Use of library software	None of the libraries visited used any

Observation	Finding
Common attitude	Record-keeping is accepted as routine; there was no expectation of an affordable alternative

2.4 Why Existing Software Was Not the Answer

The obvious response to a paper-register problem is to install a library management system. The team therefore compared the established options. Evergreen, SLiMS, eGranthalaya, LibSys and MyLibrary all offer cataloguing and circulation, and several are open source. The comparison, presented in full in Chapter 6, showed a consistent pattern: these systems are designed for institutional, government or networked libraries, and are heavier, more complex to set up, or more costly to adopt than a small panchayat library can manage.

The practical constraint is the computer itself. The machines available at these libraries are low-end PCs. Software that expects a server, a database engine, a web stack or a dedicated technical administrator is bulky for such machines and difficult for such staff. Several existing systems also assume a well-organised, consistently categorised collection, and give little room to enter or edit records in the loose, irregular way that a small, long-established collection often requires.

2.5 Why Java, Swing and CSV?

The choice of technology was shaped by two forces. The first is academic: the Object Oriented Technology Lab requires the demonstration of basic and intermediate Object-Oriented programming features, and Java is the language of the course. The second is practical: a desktop Swing application ships with the Java runtime, needs no browser, and can run offline, which suits libraries that may have unreliable connectivity. CSV text files remove the need for a database server, can be opened in any text editor or spreadsheet for inspection, and can be copied as backups using ordinary file operations.

The team accepts the trade-offs of this decision, which are discussed honestly in Chapter 26: local file storage is simpler to deploy but less suited to several computers working on the same data at once, and a Java 21 runtime has to be present on the machine that runs the JAR.

2.6 Motivation Summary

- **Personal motivation:** a real, avoidable dispute caused by a lost return record.
- **Social motivation:** rural libraries in Goa deserve the same record integrity that larger libraries enjoy.
- **Economic motivation:** the target libraries cannot fund commercial systems or technical administrators.
- **Technical motivation:** a small, dependency-free application can run on the hardware that these libraries already have.
- **Academic motivation:** the problem naturally exercises encapsulation, abstraction, inheritance, polymorphism, interfaces, exceptions, collections and event-driven design.

3. Problem Statement

Small and rural libraries in Goa maintain their records in handwritten registers. Registers are slow to search, easy to damage, impossible to back up in practice, and open to accidental error and unauthorised alteration. As a result it is difficult to confirm whether a book is available, to check whether a member is eligible to borrow, to trace a transaction, to calculate overdue charges consistently, or to recover when records are lost or damaged. Commercial and open-source library software exists, but it is generally too costly, too complex, too heavy for the available hardware, or too rigid for the way these libraries actually operate.

3.1 Problem Decomposition

Table 3: Decomposition of the problem

#	Problem	Consequence in a paper-based library
P1	Records can be lost or damaged	A return or a loan can disappear, as in the incident that began this project
P2	Records are hard to search	Finding whether a title is on the shelf or on loan takes time and relies on staff memory
P3	No consistent eligibility checks	A suspended or expired member may be lent a book; limits are applied informally
P4	Manual charge calculation	Overdue fines are calculated by hand and may be inconsistent between staff
P5	No audit trail	It is not possible to see who issued or received a book, or when
P6	No backup	A single damaged register can erase the history of years
P7	Unsuitable software	Existing systems are costly, complex or heavy for low-end PCs

3.2 Problem Statement for LibGramPocket

Individuals who lend books to friends and borrow books from them have a similar problem on a smaller scale. A lent book is easily forgotten, and the lender and borrower may later disagree about the date or the terms. There is no simple, private tool for recording who has which book, when it is due, and what, if anything, is owed.

4. Objectives

4.1 Primary Objective

To build a working, affordable and compact desktop application that lets a small library digitise its catalogue and keep every member, loan, reservation, charge and payment record in one reliable place.

4.2 Specific Objectives

1. Provide book and member catalogues with validation, duplicate detection and search.
2. Automate lending, returns and reservations with eligibility checks (membership status, expiry date, per-membership borrowing limit, book availability).
3. Calculate charges consistently from configurable rules, using exact decimal arithmetic and a pricing snapshot stored with each loan.
4. Record payments against charges and expose outstanding dues.
5. Notify staff of events and conditions that need attention, such as overdue books and expiring memberships.
6. Protect access with authentication, salted password hashes and role-based permissions.
7. Protect data with atomic writes, rolling backups, automatic recovery of unreadable files, single-instance locking and central logging.
8. Keep user interface, business rules, storage and domain entities in separate layers so the application can be tested and maintained.
9. Keep the application small enough to run on a low-end computer without a database server.
10. Apply the same architecture to build LibGramPocket for individual users.
11. Demonstrate the basic and intermediate features of Object-Oriented programming as required by the course.

4.3 Measurable Success Criteria

Table 4: Success criteria

Criterion	Target
Application size	LibGram JAR of a few hundred kilobytes (about 376 KB achieved); LibGramPocket about 276 KB
Catalogue capacity	Operate correctly with libraries of around 2,000 to 5,000 books (tested up to 5,000 books and 500 members)
Data safety	No loss of the previous file when a write fails; automatic fallback to a backup copy for unreadable files
Security	No plain-text passwords stored; permissions enforced in the service layer
Verification	A plain-JDK regression suite that covers workflows, boundaries, recovery and permissions

Criterion	Target
Deployment	Runs from a single JAR with no database server

5. Scope of the Project

5.1 In Scope

- LibGram: authentication, book catalogue, member register, lending, returns, reservations, charge calculation, payments and dues, notifications, settings, lost-book handling, member-card replacement, backup and recovery.
- LibGramPocket: per-account sign-in, owned-book catalogue, contacts, lending, borrowing, returns to lenders, itemised charges, notifications, settings and storage hardening.
- A plain-JDK automated regression suite for both applications.
- Build, run and packaging scripts for Windows and Linux/macOS.
- Documentation: this report, the READMEs and the diagrams in this document.

5.2 Out of Scope

- Multi-branch libraries and simultaneous use by several computers on one data folder.
- Cloud hosting, web access and mobile applications.
- Integration with external catalogue standards or national library networks.
- An online marketplace for exchanging books (planned as future work for LibGramPocket).
- A graphical installer and an administrator export/import wizard (listed as future ideas).

5.3 Assumptions

- A Java 21 runtime is available on the computer that runs the application.
- One staff member operates one installation at a time; the storage folder is protected by a single-instance lock.
- Rates, limits and fees are decided by each library and entered through the settings screen.
- Library data is stored locally and the library is responsible for keeping copies of its backup folder.

6. Existing System Analysis

6.1 The Manual Register System

In the panchayat libraries that were visited, the record system is a set of paper registers. A typical workflow is: the librarian writes the member, the book and the date in the register when a book is issued; the book is found on the shelf; and when it is returned the librarian looks for the earlier entry and marks it. This works while the volume is low and the same few people handle it. It fails whenever a page is damaged, an entry is skipped, a register is replaced, or two people write in different styles.

Table 5: Strengths and weaknesses of the manual system

Strengths	Weaknesses
Familiar to all staff	Entries can be omitted, misplaced or altered
No cost for software or electricity	Searching by title, author or member is slow
Works during a power cut	No backup; physical damage destroys history
No training needed	No automatic eligibility checks or charge calculation
	No reports of overdue books or expiring memberships

6.2 Comparison of Existing Software

The team compared a set of widely known library systems to decide whether an existing product could be adopted. The table below records the main features of each and the weakness that makes it less suitable for the target libraries. This comparison reflects the team's study of these products and their documentation; it is an assessment for the purpose of this project and not a formal benchmark.

Table 6: Comparison of existing library software

Software	Main features	Weakness / opportunity for LibGram
Evergreen	Library automation, cataloguing, circulation, members	Can be complex for small local libraries
LibSys	Cataloguing, circulation, reports, member management	More suited to larger library networks
SLiMS	Library management, OPAC, circulation, reports	Can be costly for small libraries
eGranthalaya	Open-source library management, cataloguing, borrowing	Requires technical setup and customisation
MyLibrary	Library automation, cataloguing, circulation, reports	Mainly designed for institutional or government libraries
Generic simple tools	Book and member management, issue-return tracking	Basic compared with modern cloud systems

6.3 Gap Analysis

The comparison reveals a gap between two extremes. At one end are paper registers, which are cheap but unreliable. At the other end are full integrated systems, which are reliable but heavy. Nothing serves the library that needs reliability without the weight. The practical requirement that the team identified is a system that is small enough for low-end PCs, simple enough for non-technical staff, flexible enough for an irregular collection, and free of server administration.

Table 7: Gap analysis

Requirement	Paper register	Typical full ILS
Affordable	Yes	Often no, or needs paid support
Runs on low-end PC	Not applicable	Often heavy
No technical administrator	Yes	Often needs one
Reliable, searchable records	No	Yes
Flexible data entry	Yes	Often rigid
Backup and recovery	No	Yes, but needs set-up
LibGram meets all of the above		

LibGram is intended to occupy this gap. It is not a replacement for a large, networked system, and the report does not claim that it is. It is a practical tool for libraries of roughly two to five thousand books that currently have no digital system at all.

> **Evidence note.** The field inquiry covered two to three libraries. If the team collected interview notes or photographs during these visits, they can be attached to this chapter as supporting evidence.

7. Proposed System

7.1 Overview

LibGram is a layered Swing application. The user interface calls services; services implement the business rules; repository interfaces isolate data access; CSV implementations persist the records; and model classes represent the domain entities. No Swing panel reads or writes a data file directly. This separation lets each rule be tested without a window and each storage detail be changed without touching the rules.

7.2 Main Capabilities

Table 8: Capabilities of LibGram

Capability	Description
Authentication	First launch asks for an administrator account; later sign-ins use salted PBKDF2 hashes; passwords must have at least eight characters
Book catalogue	Add, update, remove and view books with ISBN, title, authors, publisher, year, edition, category, language, shelf location, price and copy counts
Member register	Add, update, remove and view members with phone, email, address, membership type, join date, expiry date and status
Lending	Lend one or several books in one transaction, with a lending list and validation for every item
Returns	Return books with a condition (good, minor damage, severe damage) and automatic charge calculation
Lost books	Mark a book lost, bill the replacement price and processing fee, and handle refunds if it is found
Reservations	Reserve unavailable books; a reservation becomes Ready when a copy is returned and expires after a hold period
Charges and payments	Itemised charges with status Unpaid, Partial, Paid or Waived; payments by cash, UPI, card or other
Notifications	Event and condition alerts with severity levels, paging, dismissal and navigation to the related screen
Settings	Library rules: loan period, penalty rates, fees, membership types, borrowing limits, alert thresholds
Reliability	Atomic writes, daily rolling backups, recovery from backup copies, single-instance lock and central logs

7.3 LibGramPocket

LibGramPocket applies the same layers to a personal book collection. Each account has its own private folder of CSV files. A person records the books they own, their contacts, the books they have lent to contacts, and the books they have borrowed from contacts. Optional pricing terms (a base charge, a rent per day, week or month, and a daily penalty with a cap) allow a person

who wishes to charge a small fee to do so, and the pricing terms in force at the time of the loan are stored with the loan.

7.4 Design Principles

- **Small and dependency-free:** only the Java standard library is used; no third-party JAR is required.
- **Layered:** ui → services → data → model, with services depending on repository interfaces.
- **Defensive:** every file read tolerates damage, every write is atomic, and every failure is logged.
- **Transparent:** data is in plain text that a person can open and inspect.
- **Configurable:** library rules live in settings, not in code.
- **Testable:** business rules are pure or nearly pure and can be tested outside the user interface.

8. Stakeholders and Roles

8.1 Stakeholders

Table 9: Stakeholders

Stakeholder	Interest
Library administrator	Sets up the library, manages the catalogue, members, settings and accounts, and is responsible for backups and recovery
Library staff	Perform daily circulation: lending, returns, reservations, dues and payments
Library members	Indirect users who benefit from accurate records, fewer disputes and faster service
Individual book owners	Direct users of LibGramPocket
Project team	Develop, test and maintain the software
Project guide and evaluators	Review the work against the course requirements

8.2 Roles and Permissions in LibGram

The user model defines two roles, ADMIN and STAFF. A PermissionPolicy service decides what each role may do, and it also requires the account to be active. The same policy is used by the user interface (to hide or disable options) and by the services (to refuse operations), so that a restriction does not depend on the screen alone.

Table 10: Permission matrix (from PermissionPolicy)

Operation	ADMIN	STAFF	Inactive / none
Manage catalogue (add, update, remove books)	Yes	No (view only)	No
Manage members (add, update, remove)	Yes	No (view only)	No
View members	Yes	Yes	No
Circulate (lend, return, reservations)	Yes	Yes	No
Open settings	Yes	No	No
Manage accounts	Yes	No	No

In the user interface, a staff user sees the catalogue and member screens in a view-only form; the tooltip on a disabled member option states that only an administrator can change members. A permission-matrix test calls the policy directly to confirm these rules.

9. Requirements

9.1 Functional Requirements

Table 11: Functional requirements

ID	Requirement	Module
FR-01	The system shall create an administrator account on first launch and authenticate users on later launches	Authentication
FR-02	The system shall add, update, remove and view books and shall reject a duplicate ISBN	Catalogue
FR-03	The system shall search books by title, author, ISBN, publisher, category, language or year, with an Available-only filter	Catalogue
FR-04	The system shall add, update, remove and view members and shall reject a duplicate email or phone number	Members
FR-05	The system shall support membership types with different borrowing limits, lengths and fees	Members
FR-06	The system shall lend books only to active, unexpired members who are under their borrowing limit	Lending
FR-07	The system shall lend only books that have an available copy and shall update availability atomically with the loan	Lending
FR-08	The system shall record a return with a condition and update availability	Returns
FR-09	The system shall calculate base charge, rent, overdue fine, damage and processing charges from the loan's pricing snapshot	Billing
FR-10	The system shall mark a book lost after the configured period or on request and bill the replacement price and fee	Lost books
FR-11	The system shall create reservations, promote them to Ready when a copy becomes available, and expire them after the hold period	Reservations
FR-12	The system shall record payments against charges and compute the outstanding balance	Payments
FR-13	The system shall raise notifications for key events and conditions and avoid duplicate alerts	Notifications
FR-14	The system shall allow an administrator to change library rules in settings	Settings
FR-15	The system shall restrict operations by role at the service layer	Security
FR-16	The system shall back up data files daily and recover an unreadable file from its backup	Reliability
FR-17	The system shall prevent a second process from using the same storage folder	Reliability
FR-18	The system shall write diagnostic information to log files	Logging

9.2 Non-Functional Requirements

Table 12: Non-functional requirements

Quality	Requirement
Footprint	The application shall be small enough to run on a low-end PC; the JAR is under 400 KB
Performance	Common operations shall respond without noticeable delay for libraries of up to about 5,000 books and 500 members
Reliability	A failed write shall leave the previous file intact; a damaged file shall not crash the application
Security	Passwords shall be stored only as salted PBKDF2 hashes; permissions shall be enforced in services
Maintainability	Layers shall be separated; business rules shall not depend on Swing or on concrete files
Testability	Rules shall be testable by plain Java programs without a graphical session
Usability	Standard Swing components, labelled controls, search in every catalogue, and readable messages
Portability	The application shall run wherever a Java 21 runtime is available

The README documents a performance smoke test that uses much larger datasets (10,000 books, 2,000 members, 20,000 loans and 30,000 charges). These are stress-test sizes; they are not a promise of speed on every computer, and the capacity the team verified in field conditions is 5,000 books and 500 members.

9.3 LibGramPocket Requirements

- Each account shall have a private data folder; one account shall not see another's data.
- A user shall maintain owned books and contacts, and record lends and borrows with due dates.
- Optional pricing terms shall be remembered separately for lending and for borrowing.
- Alerts for books due and amounts owed shall run periodically and shall be switchable in settings.
- The application shall start with a consistent inventory: over-lent copies are repaired to zero available.

10. Feasibility Study

10.1 Technical Feasibility

Java 21 and Swing are part of a standard Java distribution; no additional framework is required. CSV storage is implemented with the standard file API, including atomic file replacement. Cryptographic password hashing uses the standard PBKDF2WithHmacSHA256 algorithm. The project therefore relies on technology that is mature, documented and free, and the team had the skills required through the OOTL course.

10.2 Economic Feasibility

The software has no licence cost and needs no database server, so the only cost to a library is the computer it already owns. The development cost was the time of the two team members. This is the central point of the project: affordability is a design requirement, not an accident.

10.3 Operational Feasibility

Library staff are used to registers, and the application's menus follow the same vocabulary: Book-keeping, Members, Lend a book, Fees and Payments, and Recover a book. The operational conditions are that the library has a computer running a Java 21 runtime, that staff receive brief training, and that one installation is used at a time.

10.4 Schedule Feasibility

The project was carried out in iterative phases over the semester and was completed in time for the mini-project demonstration. The phase structure is described in Chapter 13.

10.5 Legal and Ethical Feasibility

The software stores personal data about library members, namely name, phone, email and address. It stores this data locally and does not transmit it. Section 27.4 discusses the privacy practices that a library should follow. The demonstration data used in the project is fictional.

Table 13: Feasibility summary

Dimension	Rating	Reason
Technical	High	Standard Java and Swing; no external dependencies
Economic	High	No licence or server cost
Operational	Medium to High	Familiar vocabulary; requires Java runtime and brief training
Schedule	High	Iterative phases completed within the course period
Legal / ethical	High	Local storage; fictional demo data; privacy guidance provided

11. Brainstorming and Idea Generation

Brainstorming began with the incident described in Chapter 2 and widened into a list of questions: What would have prevented the dispute? What would a librarian need on the counter? What would make the data safe? What could run on the available PCs? The ideas that emerged were grouped into themes.

Figure 3.1: Brainstorming Mind Map of LibGram

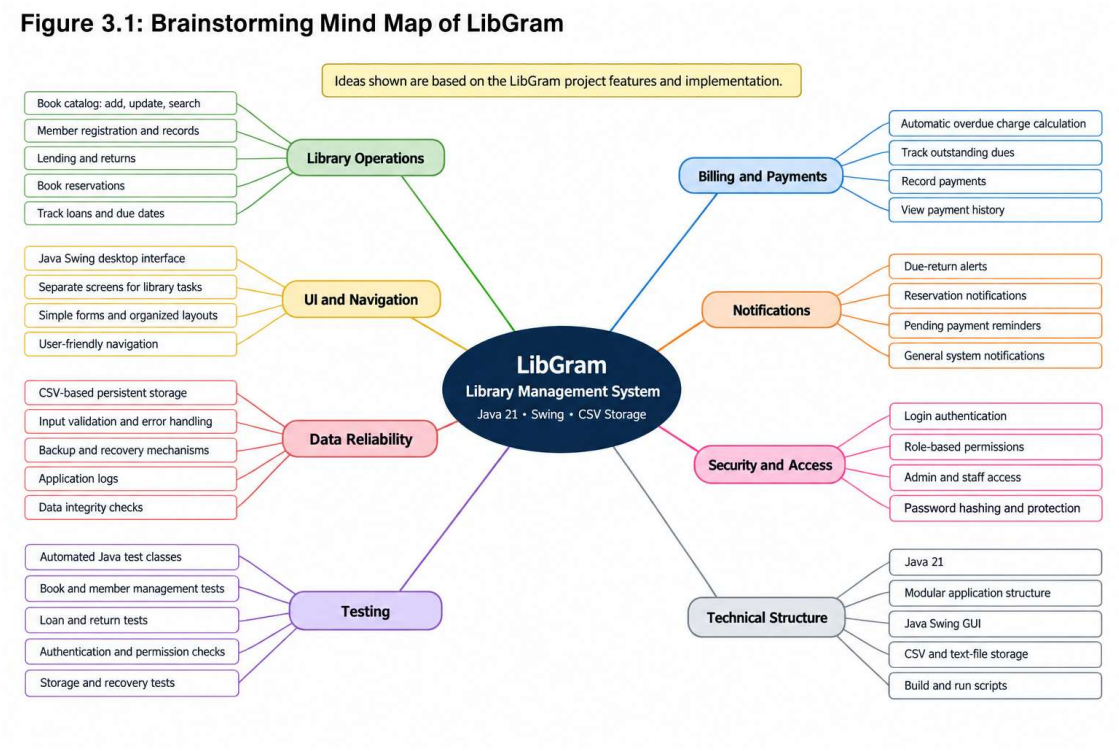


Figure 1: Brainstorming and idea-generation map for LibGram

11.1 Idea Themes

Table 14: Idea themes generated during brainstorming

Theme	Ideas
Library operations	Book catalogue, member registration, lending, returns, reservation management
Billing	Automatic charge calculation, dues tracking, payment recording
Notifications	Alerts for loans, returns, reservations and billing activities
Security	Login authentication, role-based permissions, password protection
Data reliability	CSV-based storage, automatic backups, recovery mechanisms, application logs
User interface	A Java Swing desktop interface with separate screens for different library functions
Testing	Automated test cases to verify functionality, permissions, data handling and recovery

Theme	Ideas
Individual users	A personal edition for people who lend and borrow books (later named LibGramPocket)

11.2 How the Ideas Were Evaluated

Each idea was evaluated against four questions: Does it address the incident or the field findings? Can it run on a low-end PC? Can it be built and tested within the course? Does it exercise the Object-Oriented concepts that the course requires? Ideas that answered yes to all four were accepted. Ideas that failed on weight, complexity or schedule were rejected, as recorded in Chapter 12.

12. Accepted and Rejected Ideas

12.1 Accepted Ideas

Table 15: Accepted ideas

Idea	Reason for acceptance
Java Swing desktop application	Provides a graphical interface, runs offline, needs no browser, and suits the course requirements
CSV and text-file storage	Local storage without a database server; human-readable and easy to back up
Book and member management	Core of any library; replaces the register
Lending, returns and reservations	Manages circulation and removes the dispute that started the project
Billing and payment tracking	Supports overdue charges and payment records consistently
Authentication and role-based access	Protects records and distinguishes administrators from staff
Backup and recovery	Protects stored data and addresses the loss-of-records problem
Automated testing	Provides verification and supports the maintainability goal
LibGramPocket for individuals	Reuses the architecture for a second audience

12.2 Rejected Ideas

Table 16: Rejected or not-selected ideas

Idea	Reason for not selecting
Web-based application	Needs a server or hosting and a connection; a desktop application matched the implementation scope and the offline setting
Cloud-based data storage	Depends on connectivity and recurring cost; local storage was more suitable for the intended libraries
Separate database server	Adds installation, configuration and administration that small libraries cannot support; CSV files avoided it
Manual record maintenance	Would not provide the automation, eligibility checks or reliability that the project aims for
Command-line-only interface	A graphical interface was preferred for ease of use by non-technical staff
Adopting an existing product	Existing products were too costly, complex or heavy for the target libraries (Chapter 6)

12.3 Decision Rationale

The most consequential decision was to choose a desktop application with plain file storage rather than a web or database-backed design. The trade-off was explicit. The team gave up simultaneous multi-computer access in exchange for a system that a librarian can start by

double-clicking a file, with data that can be read and copied with ordinary tools. Because the libraries concerned have a single circulation desk and a single computer, that trade was judged to be the right one.

13. Development Methodology

LibGram was developed with an iterative, modular approach. The application was split into modules for book management, member management, lending, returns, reservations, billing, notifications and security. Each module was implemented, tested, and then integrated into the running application before the next was started, so that the product was always in a working state.

13.1 Stages

1. **Requirement analysis:** identify the functional requirements from the field inquiry and the competitor comparison.
2. **System design:** plan the application structure, data models, storage mechanism and user interface.
3. **Implementation:** build the application in Java 21 and Swing, with CSV text files for storage.
4. **Testing:** use automated plain-Java test programs to verify functionality, authentication, permissions and data handling.
5. **Debugging:** identify and correct defects found during testing.
6. **Integration and hardening:** combine the modules into a complete application and add recovery, locking, logging and packaging.

13.2 Phases of LibGram

The README and the test names show a phase-oriented development. The project reached Phase 9, which hardened the application for release. The test suite retains tests that are named after the phases, which makes the history visible in the code.

Table 17: Development phases of LibGram

Phase	Focus	Evidence in the repository
1	Foundation: sign-in, main window, navigation and layout	PhaseOneSmokeTest; Theme, MainFrame, LoginPanel
2 to 5 and 7	Catalogue, members, lending, returns, reservations, dues and recovery workflows (lost books, card replacement)	CatalogService, MemberService, LendingService, ReturnService, ReservationService, LostBookService, RealDuesProvider and their tests
6	Charges, billing and payments	ChargeCalculator, ChargeService, PaymentService, PhaseSixBillingTest, PhaseSixWorkflowTest
8	Notifications and alert scanning	NotificationService, AlertScanner, listener classes, PhaseEightNotificationTest
9	Hardening: recovery, backups, locking, logging, packaging, demo data	PhaseNineHardeningTest, RepositoryRecoveryTest, StorageLock, AppLog, build scripts

> **Note.** Phases 2 to 5 and 7 are grouped because the repository records them through the module services and their tests rather than through phase-named tests. The grouping reflects the source structure, not a dated timeline.

13.3 Phases of LibGramPocket

Table 18: Development phases of LibGramPocket (from its README)

Phase	Focus
1	Sign-in and private per-account storage
2	Owned-book create, read, update, delete and search
3	Contacts
4	Settings and remembered preferences
5	Lending, returns, itemised charges, payments and inventory repair
6	Borrowing from contacts and returning to lenders
7	Per-account notifications, due and owed alerts, menu badges and a periodic scanner
8	Storage hardening, password change, About screen and packaging

13.4 Tools and Environment

Table 19: Tools used

Tool	Use
Java 21 (JDK)	Language, compiler, runtime and the jar tool
Swing (javax.swing)	Graphical user interface, including the Nimbus look and feel
java.nio.file and java.nio.channels	Atomic file replacement and file locking
javax.crypto (PBKDF2WithHmacSHA256)	Password hashing
Plain-JDK test mains	Regression testing without an external framework
Batch and shell scripts	Build, run and test on Windows and Linux/macOS
Git and GitHub	Version control and hosting of the project repository
Visual Studio Code	Source editing

14. System Architecture

14.1 Layered Architecture

The source is organised into four packages whose dependencies run in one direction: **ui → services → data → model**. The service layer is written against repository interfaces, not against concrete CSV classes, so the rules do not know how or where the data is stored.

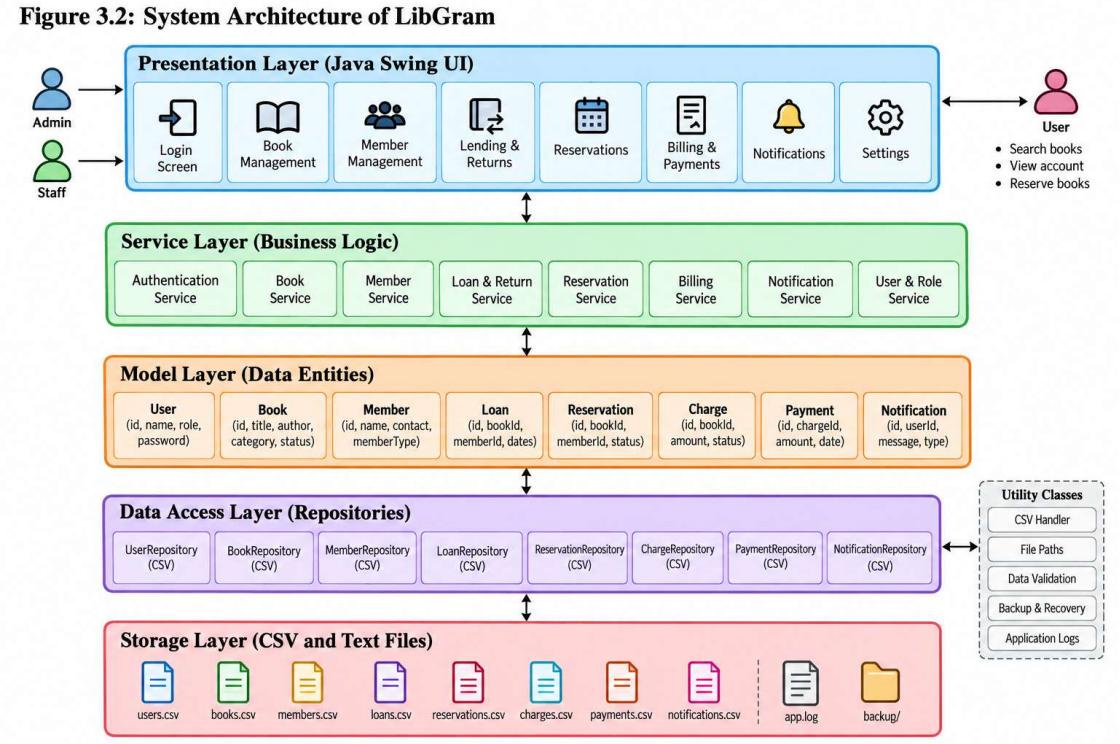


Figure 2: System architecture of LibGram

Table 20: Layers and responsibilities

Layer	Package	Responsibility
Presentation	ui	Swing windows, panels and dialogs; collects input and shows results; never touches files
Business logic	services	Validation, business rules, permission checks, charge calculation, event publication and alert scanning
Data access	data	Repository interfaces, CSV implementations, CSV utilities, storage paths, storage lock and logging
Domain	model	Immutable records and enumerations that represent library entities

14.2 Size of the Code Base

Table 21: Code base metrics (from the repository)

Item	LibGram	LibGramPocket

Item	LibGram	LibGramPocket
Source files (src)	108	79
Source lines (src)	About 8,860	About 8,890
Test files (test)	20	27
Test lines (test)	About 1,680	About 2,590
Model classes and enums	20	14
Service classes and interfaces	42	22
Data classes and interfaces	22	16
UI classes	23	26

14.3 Start-up Sequence

The entry point, Main, runs on the Swing Event Dispatch Thread. It installs the Nimbus look and feel, resolves the storage directory (a development folder can be chosen with the --storage argument), configures logging, acquires the single-instance storage lock, and registers a shutdown hook that releases the lock. It then builds the object graph by hand: CSV repositories are created, services are created from them, and notification listeners are registered with the services that publish events. The login panel is shown, and the main frame opens after a successful sign-in.

```
storage lock -> logging -> repositories -> services -> listeners -> login -> main window
```

Constructing the graph explicitly in one place (rather than with a framework) keeps the dependencies visible, which is helpful for a course project and sufficient for an application of this size.

14.4 Event-Driven Collaboration

Services announce what happened through small listener interfaces such as LoanEventListener, ReturnEventListener, ReservationEventListener, BillingEventListener, BookChangeListener, MemberChangeListener, LostBookEventListener and SettingsChangedListener. Notification listeners implement these interfaces and turn events into notifications. This is the Observer pattern. A lending service does not know that notifications exist; it only announces that a loan happened.

Table 22: Listener interfaces and their notification implementations

Event interface	Publisher	Notification listener
LoanEventListener	LendingService	LoanNotificationListener
ReturnEventListener	ReturnService	ReturnNotificationListener
ReservationEventListener	ReservationService	ReservationNotificationListener
BillingEventListener	ChargeService, PaymentService	BillingNotificationListener

Event interface	Publisher	Notification listener
BookChangeListener	BookService	BookNotificationListener
MemberChangeListener	MemberService	MemberNotificationListener
LostBookEventListener	LostBookService	LostBookNotificationListener
SettingsChangedListener	SettingsService	Refreshes dependent screens

15. System Design Diagrams

This chapter presents the diagrams prepared for the project: the use-case diagram, the activity diagram and the sequence diagram for lending, the data-flow diagram and the conceptual class diagram. Each diagram is followed by a short explanation so that it can be read without further context.

15.1 Use-Case Diagram

Figure 3.3: Use-Case Diagram of LibGram

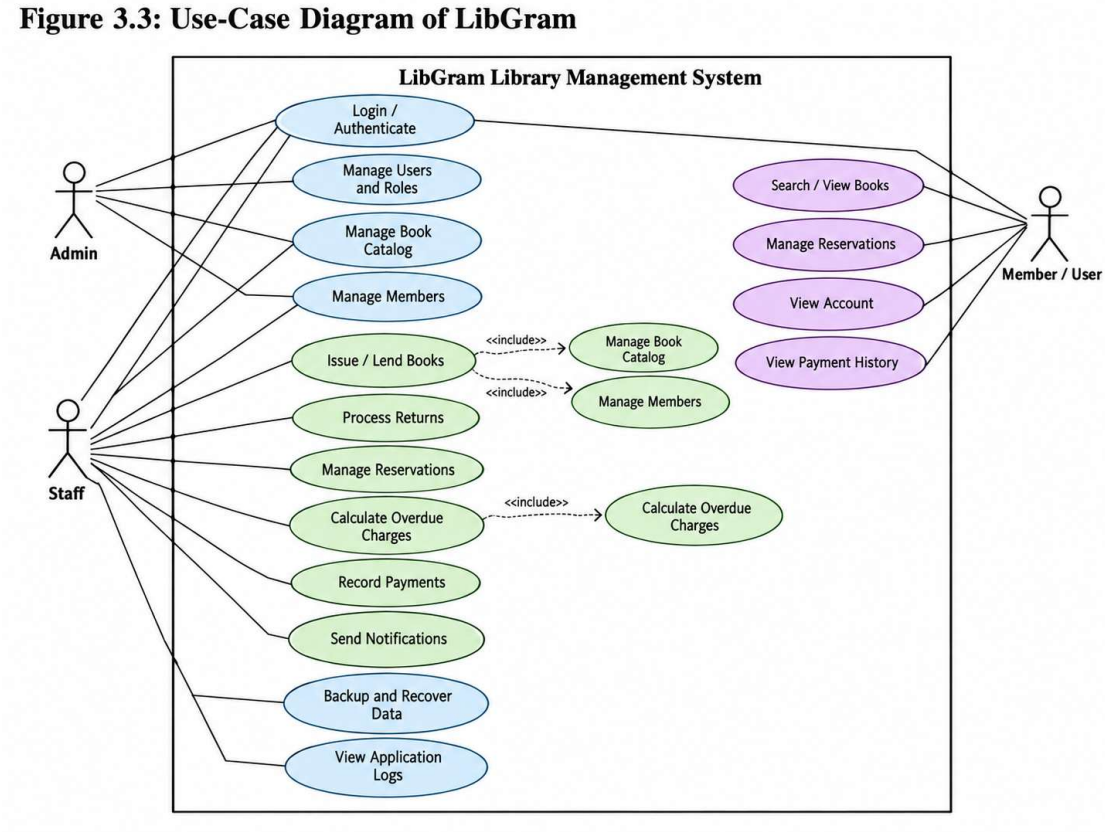


Figure 3: Use-case diagram of LibGram

The actors are the library administrator and authorised library staff. The system boundary is the LibGram application. The principal use cases are authenticate, manage books, manage members, issue (lend) a book, return a book, manage reservations, review charges, record payment, review notifications and configure settings. Configuration, catalogue changes and member changes are limited to administrators, while circulation is available to both roles.

Table 23: Use cases and primary actor

Use case	Primary actor	Supporting service
Authenticate	Administrator, Staff	AuthService
Manage books	Administrator	BookService, CatalogService
Manage members	Administrator	MemberService, MembershipService

Use case	Primary actor	Supporting service
Issue a book	Administrator, Staff	LendingService
Return a book	Administrator, Staff	ReturnService
Manage reservations	Administrator, Staff	ReservationService
Review charges and collect payment	Administrator, Staff	ChargeService, PaymentService
Mark a book lost	Administrator, Staff	LostBookService
Review notifications	Administrator, Staff	NotificationService
Configure settings	Administrator	SettingsService

15.2 Activity Diagram: Lending

Figure 3.4: Activity Diagram – Lending (Issue Book)

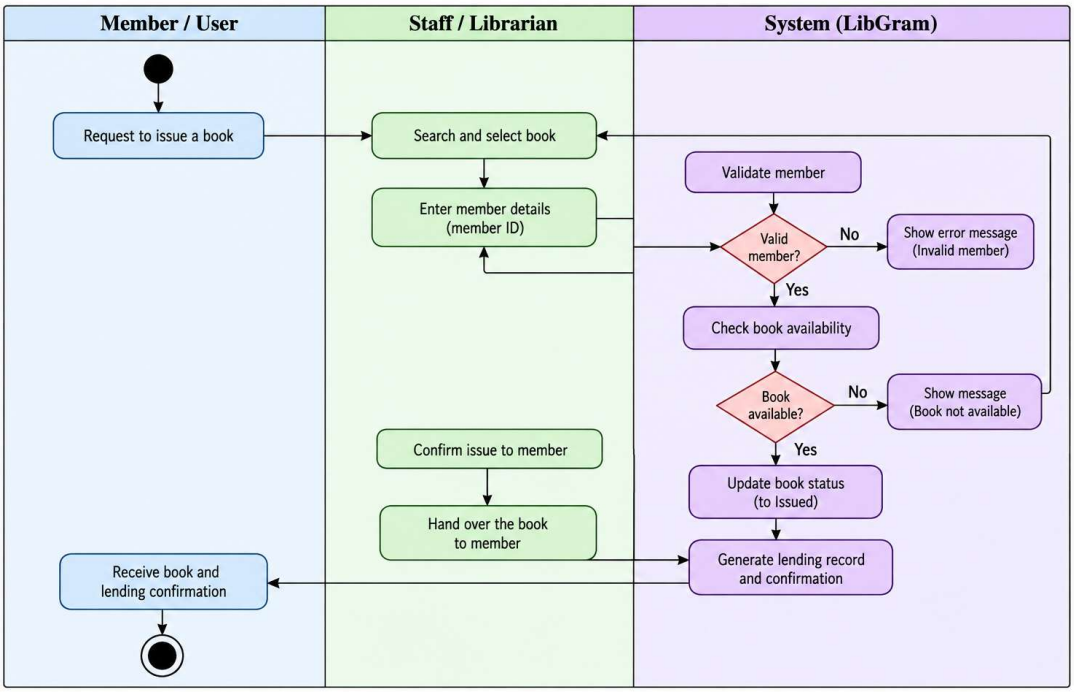


Figure 4: Activity diagram for the lending workflow

The lending activity begins when staff select a member and add one or more books to a lending list. The system then validates the member and each book. If any check fails, the whole transaction is refused with every failure reported together, and nothing is written. If all checks pass, the loan records are created, the available copies are reduced, the data is saved atomically, and a confirmation is shown.

Validation checks performed before a loan is created

1. The member exists, is Active, and the membership has not expired.
2. The member has no overdue books that must be returned first.
3. The member has no unpaid dues.

4. The number of books after this lending does not exceed the borrowing limit of the membership type.
5. Each book exists, appears only once in the list, and has an available copy.
6. The member does not already hold an active loan for the same title.
7. The due date lies between today and the maximum loan date allowed by settings.

15.3 Sequence Diagram: Lending

Figure 3.5: Sequence Diagram – Lending (Issue Book)

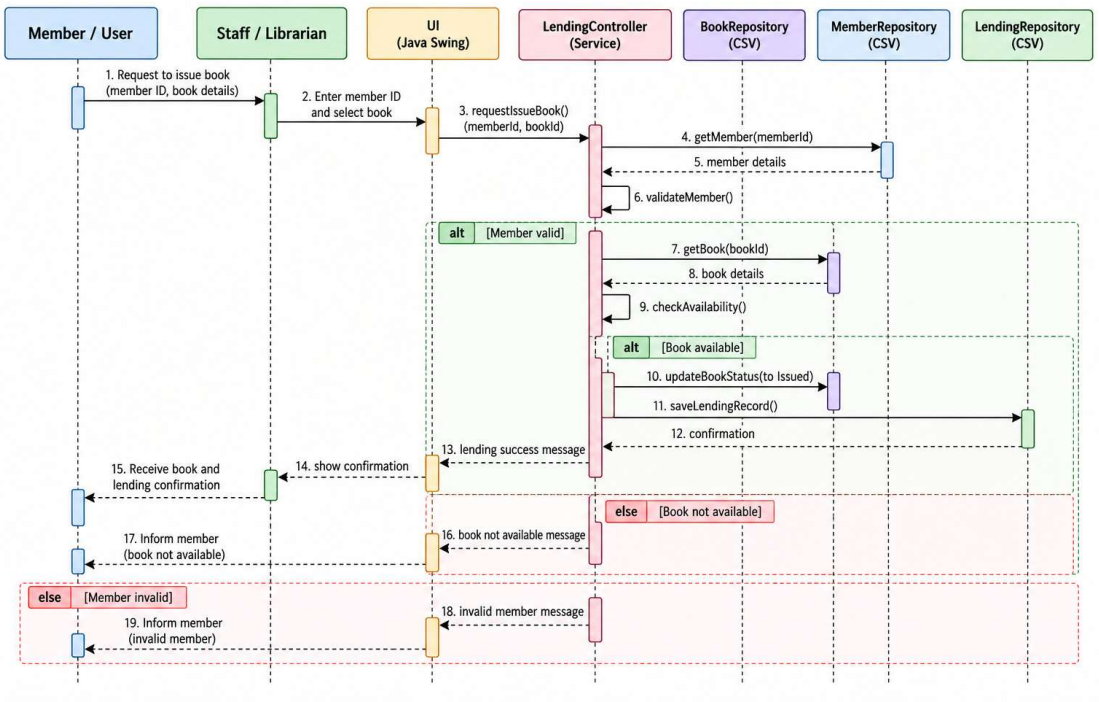


Figure 5: Sequence diagram for lending a book

The sequence diagram shows the order of messages. The administrator selects a member and a book in the lending screen. The screen asks LendingService to lend. The service checks permissions and applies LoanRules and related checks, reads the member, book and loan records through the repository interfaces, writes the new loan and the reduced availability through the CSV repositories, announces a loan event to the listeners, and returns success or a validation error to the screen. The repository writes use the atomic replacement technique described in Chapter 22.

15.4 Data-Flow Diagram

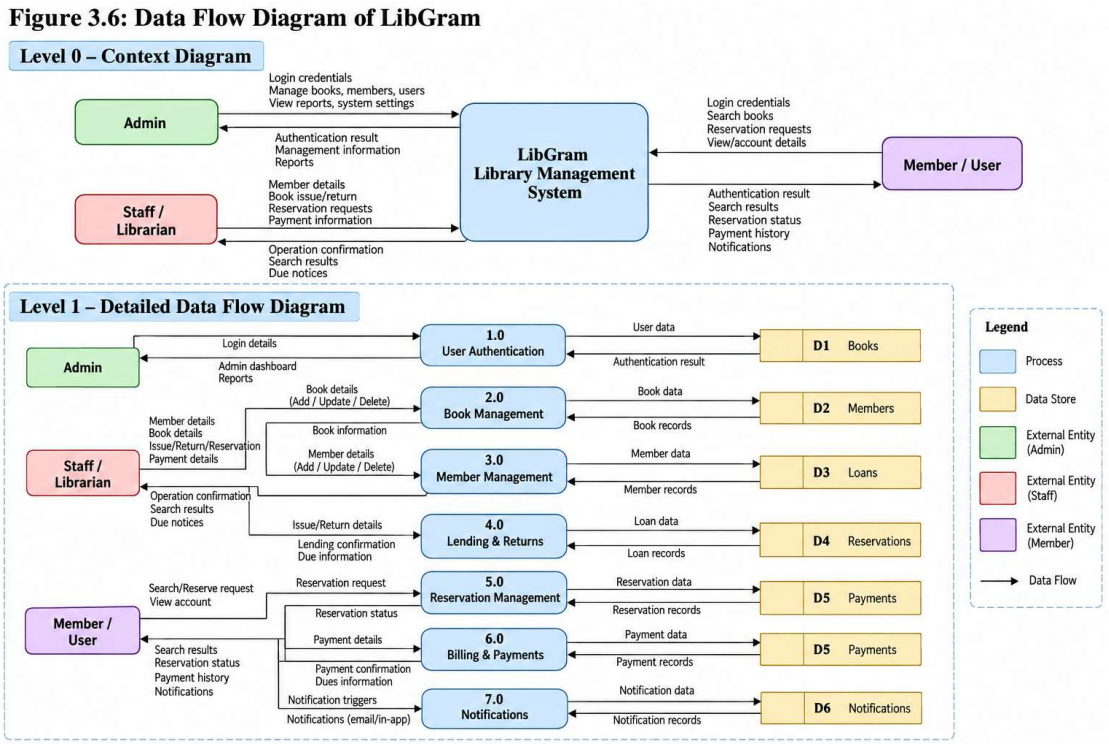


Figure 6: Data-flow diagram of LibGram

At context level, the administrator interacts with a single process, the LibGram system. The process reads and writes the data stores for users, books, members, loans, reservations, charges, payments, notifications and settings. Two operational stores support reliability: the log files and the rolling backups. Data does not leave the computer.

Table 24: Data stores in the data-flow diagram

Store	Contents
users	Accounts, roles, salts, password hashes, active flag
books	Catalogue records and copy counts
members	Member register and membership status
loans	Active and returned loans with pricing snapshot
reservations	Reservation queue and status
charges	Itemised charges and their status
payments	Payments recorded against charges
notifications	Alerts and events with severity and dismissal state
settings	Library rules and defaults
logs, backups	Diagnostics and rolling copies of the data files

15.5 Conceptual Class Diagram

Figure 3.7: Conceptual Class Diagram of LibGram

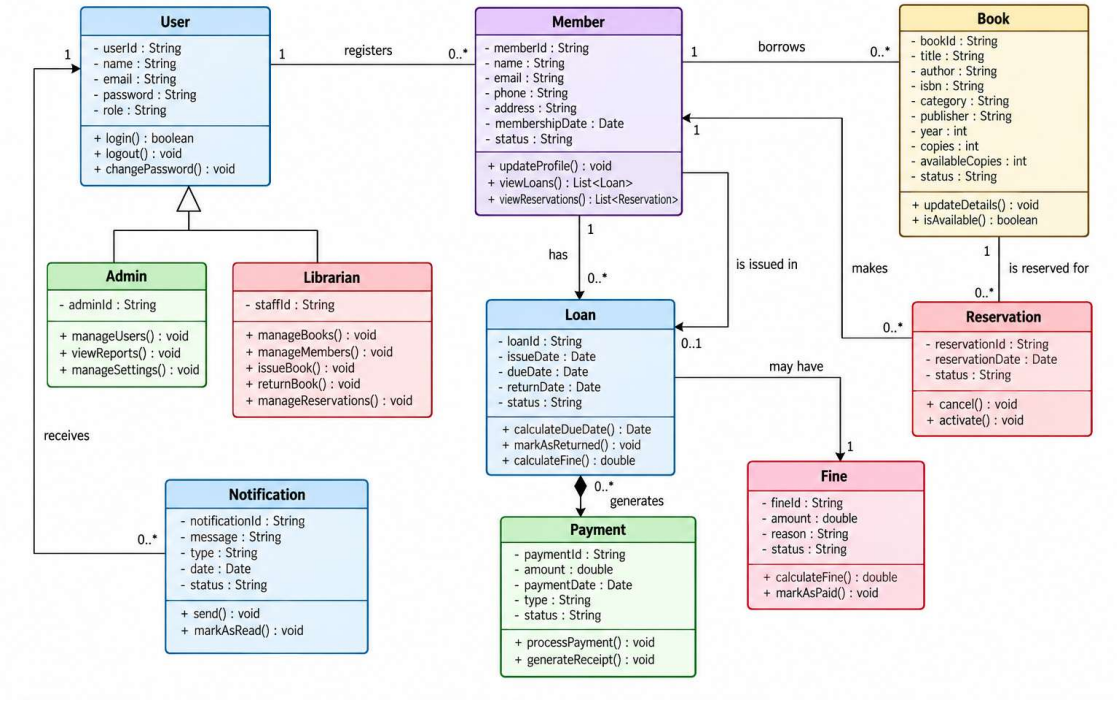


Figure 7: Conceptual class diagram of LibGram

The conceptual class diagram shows the domain entities and their relationships. A Member borrows Books through Loans; a Loan can produce Charges; a Payment settles one or more Charges; a Reservation links a Member to a Book; Notifications refer to the entities that caused them; and Settings hold the rules. The classes are implemented as immutable Java records, discussed in Chapter 16.

15.6 Design Decisions Visible in the Diagrams

- Loans store a **title snapshot** and a **pricing snapshot** so that later edits to a book or to the pricing rules do not rewrite history.
- Charges are separate records from loans so that a loan can produce several itemised lines (rent, fine, damage, processing).
- Payments are linked to charges, not to loans, so that a partial payment is represented exactly.
- Notifications carry a reference id and a deduplication key so that a repeating scanner does not create duplicates.

16. Data Model and File Inventory

16.1 Domain Entities

Entities are defined as Java records, which are immutable data carriers. Monetary values use BigDecimal with two decimal places, never floating-point numbers. The principal entities and their fields are listed below.

Table 25: Entity: Book

Field	Meaning
bookId, isbn	Internal identifier and ISBN (unique)
title, authors, publisher, publicationYear, edition	Bibliographic details
category, language, shelfLocation	Classification and where the book is shelved
price	Replacement price, used for lost and severely damaged books
totalCopies, availableCopies	Copy counts; availability changes with lending and returns
dateAdded	Date the record was created

Table 26: Entity: Member

Field	Meaning
memberId, name	Identifier and name
phone, email, address	Contact details; phone and email must be unique
membershipType	INDIVIDUAL, FAMILY, SENIOR or TEMPORARY
joinDate, expiryDate	Membership period
status	ACTIVE or SUSPENDED
createdAt, updatedAt	Audit timestamps

Table 27: Entity: Loan

Field	Meaning
loanId, bookId, memberId	Identifiers
bookTitleSnapshot	Title at the time of lending
issueDate, dueDate, returnDate	Loan dates; for a lost loan the return-date field stores the date it was marked lost
status	ACTIVE, RETURNED or LOST
baseCharge, rentCharge, overdueFine	Charge amounts
returnCondition	GOOD, MINOR_DAMAGE or SEVERE_DAMAGE
issuedBy, receivedBy	Staff accountable for the transaction

Field	Meaning
pricingSnapshot	Rent and penalty terms in force when the loan was created

Table 28: Entity: Reservation, Charge, Payment, Notification, User, Settings

Entity	Fields
Reservation	reservationId, bookId, bookTitleSnapshot, memberId, reservedAt, status (PENDING, READY, COLLECTED, CANCELLED, EXPIRED), readyAt, expiresAt, feeChargeId
Charge	chargeId, memberId, loanId, type, description, amount, waivedAmount, paidAmount, status (UNPAID, PARTIAL, PAID, WAIVED), createdAt; balance() = amount minus waived minus paid
Payment	paymentId, memberId, chargeId, amount, method (CASH, UPI, CARD, OTHER), receivedBy, paidAt
Notification	notificationId, timestamp, context, type, severity, message, refId, dedupKey, dismissed
User	userId, username, displayName, role (ADMIN, STAFF), salt, passwordHash, active, createdAt
Settings	Key-value rules with a schema version; defaults come from SettingsDefaults

16.2 Enumerations

Table 29: Enumerations used in the model

Enumeration	Values
MembershipType	INDIVIDUAL, FAMILY, SENIOR, TEMPORARY
MemberStatus	ACTIVE, SUSPENDED
LoanStatus	ACTIVE, RETURNED, LOST
ReservationStatus	PENDING, READY, COLLECTED, CANCELLED, EXPIRED
ChargeType	BASE, RENT, OVERDUE_FINE, LOST_BOOK, DAMAGED_BOOK, PROCESSING_FEE, MEMBERSHIP_REGISTRATION, MEMBERSHIP_RENEWAL, CARD_REPLACEMENT, RESERVATION, REFUND
ChargeStatus	UNPAID, PARTIAL, PAID, WAIVED
PaymentMethod	CASH, UPI, CARD, OTHER
ReturnCondition	GOOD, MINOR_DAMAGE, SEVERE_DAMAGE
NotificationSeverity	INFO, WARNING, ALERT
NotificationContext	BOOKKEEPING, MEMBERS, LENDING, RECOVERY (the menu section an alert belongs to)

16.3 Storage Layout

```
storage/
  users.txt            accounts and password hashes
  books.txt            book catalogue
  members.txt         member register
  loans.txt            lending and return records
  reservations.txt   reservation queue
  charges.txt          itemised charges
  payments.txt        payment records
  notifications.txt   alerts and events
  settings.txt         library rules
  backups/             dated copies, newest five per file
  logs/                libgram-YYYY-MM-DD.log (kept 14 days)
  .libgram.lock        single-instance lock
```

16.4 Source Tree

Table 30: Project directories and files

Path	Description
src/model/	Records and enumerations
src/services/	Authentication, lending, billing, reservations and other business operations
src/data/	Repositories, CSV utilities, storage paths, storage lock and logging
src/ui/	Swing screens, panels and dialogs
src/Main.java	Application entry point
test/	Plain-JDK regression tests and runners
build.bat, build.sh	Build scripts (compile, test, package)
run.bat, run.sh	Launch scripts
LibGram.jar	Packaged application with Main as entry point
README.md	Build, run, backup and assumption notes

17. Implementation: Book Catalogue and Member Management

17.1 Book Catalogue Module

The catalogue module manages the library's collection. BookService and CatalogService provide the operations, BookCatalogPanel and BookFormDialog provide the screens, and CsvBookRepository stores the records. Only administrators may add, update or remove books; staff can view them.

Each record holds the ISBN, title, authors, publisher, publication year, edition, category, language, shelf location, replacement price, total copies and available copies. A duplicate ISBN is rejected with a dedicated DuplicateIsbnException, which lets the user interface show a precise message rather than a generic failure.

Searching

The main window has a search field with a criterion selector: All, Title, Author, ISBN, Publisher, Category, Language or Year. An Available-only checkbox restricts the list to books that have at least one copy on the shelf. Searching is performed in memory on the loaded list, which is fast for collections of a few thousand books.

Key behaviours

- Add, update and remove books through a form dialog with validation of required fields.
- Reject a duplicate ISBN.
- Maintain total and available copy counts; availability is changed only by lending, returns, lost-book handling and inventory repair.
- Publish change events so that the notification service can record book-added, book-updated and book-removed entries.
- Raise a low-stock alert when available copies fall to the configured threshold.

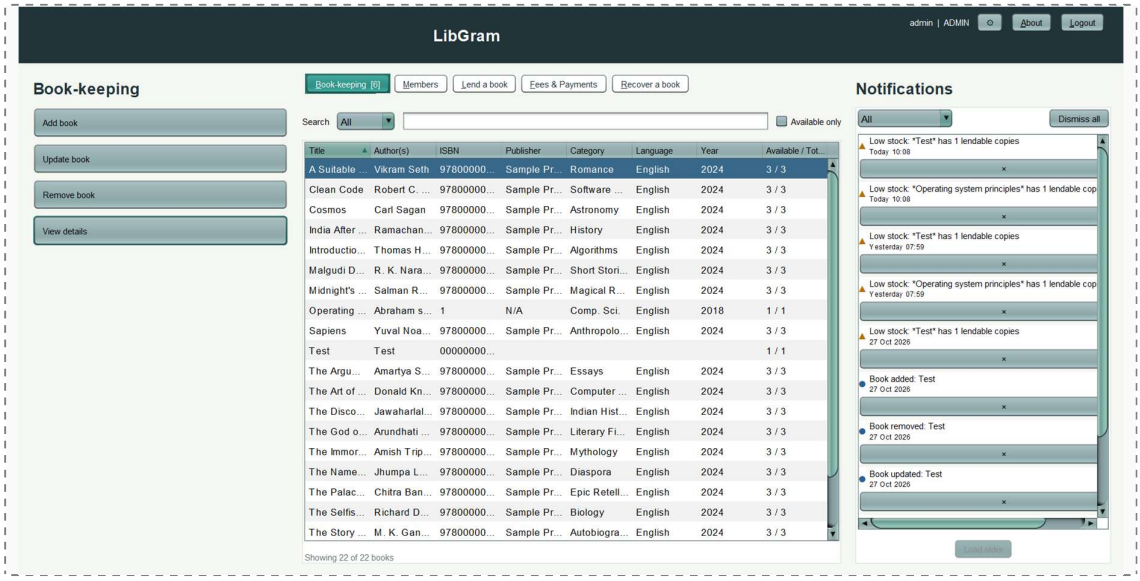


Figure 8: Book catalogue screen with search and filter

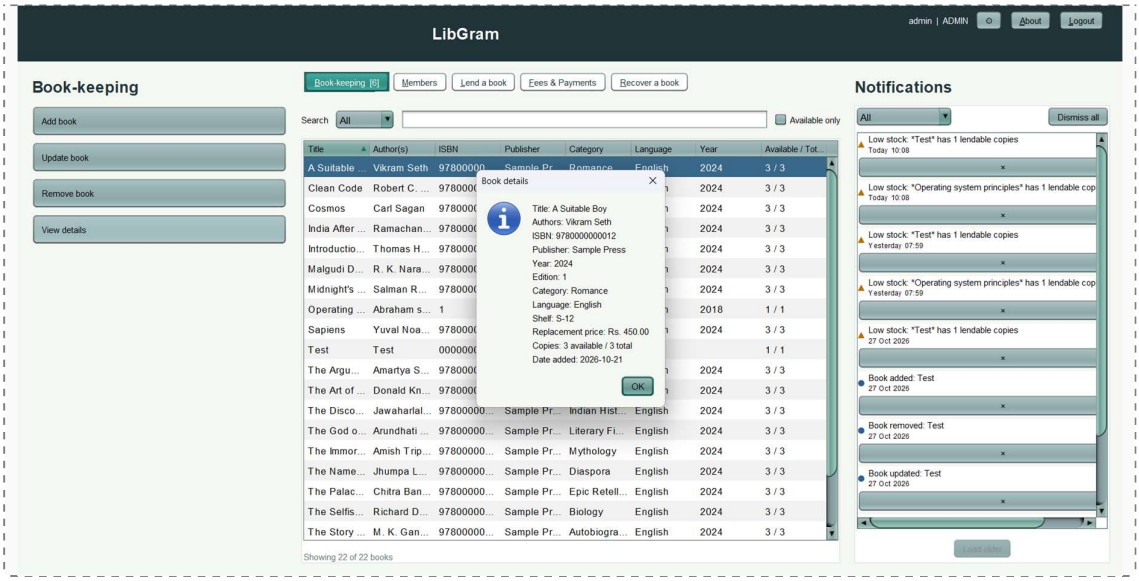


Figure 9: Add book form dialog

17.2 Member Management Module

MemberCatalogPanel, MemberFormDialog, MemberDetailPanel and MemberSearchPanel provide the member screens. MemberService and MembershipService handle the operations and the membership rules. The member detail view shows the member's currently borrowed books, loan history and dues in separate panels.

Membership types

Four membership types are defined, each with a borrowing limit, a membership length and fees that are configurable in settings. The defaults supplied in SettingsDefaults are shown below.

Table 31: Default membership terms (SettingsDefaults)

Type	Book limit	Length (days)	Registration fee (Rs.)	Annual fee (Rs.)
Individual	3	365	100.00	100.00
Family	6	365	200.00	200.00
Senior	3	365	0.00	0.00
Temporary	2	30	50.00	0.00

Validation and eligibility

Phone number and email must be unique across members; DuplicateMemberPhoneException and DuplicateMemberEmailException report violations. A member is eligible to borrow only when the status is Active and the expiry date has not passed. A suspended or expired member is refused at lending and reservation time. Renewing a membership requires that all non-renewal

dues have been paid, then extends the expiry date by the length configured for the membership type and creates a renewal charge if a fee applies.

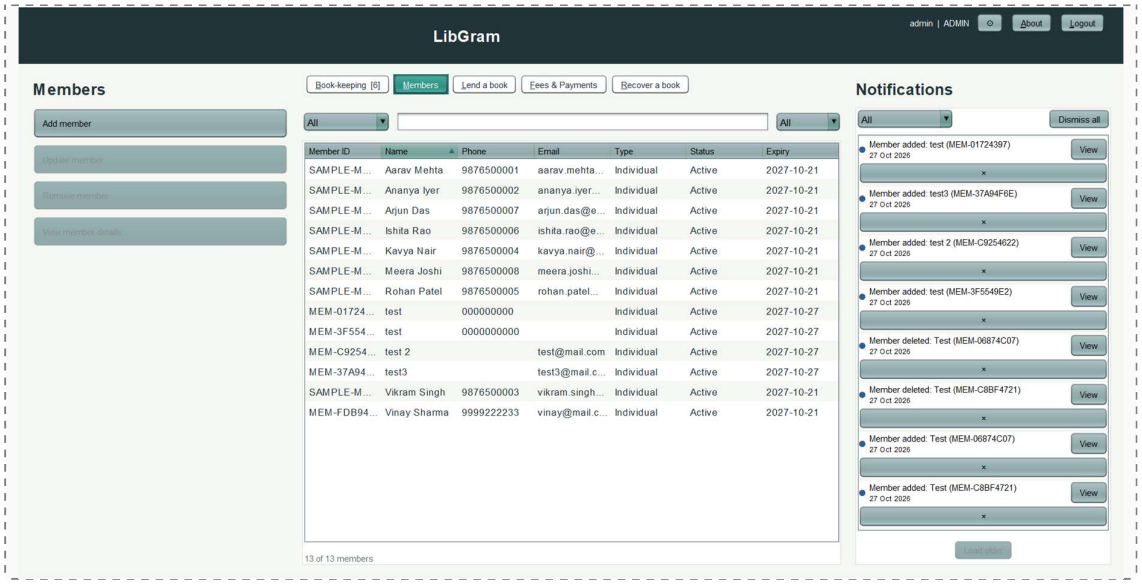


Figure 10: Member catalogue and member detail view

18. Implementation: Lending, Returns and Reservations

18.1 Lending Module

LendingPanel presents a lending list (a cart). Staff pick a member, add books and choose the due date, which defaults to the configured loan period. LendingService validates the whole request and either creates every loan or none. The validation checks are listed in Section 15.2. Failures are collected and reported together, so staff see everything that needs fixing in one message instead of discovering problems one by one.

Atomicity of a lending transaction

A lending operation changes two files: loans and books (availability). The service reads current data, applies the change in memory, and writes through the repositories with atomic replacement. Because the validations are completed before the first write, a refused lending leaves both files unchanged. The invariant tests described in Chapter 24 check that sequences of operations never leave a loan and a book availability out of step.

Rules supplied by LoanRules

Table 32: Date rules in LoanRules

Method	Meaning
daysOverdue(loan, today)	Number of days past the due date, never negative
dueSoon(loan, today)	True when the due date is today or within the next three days

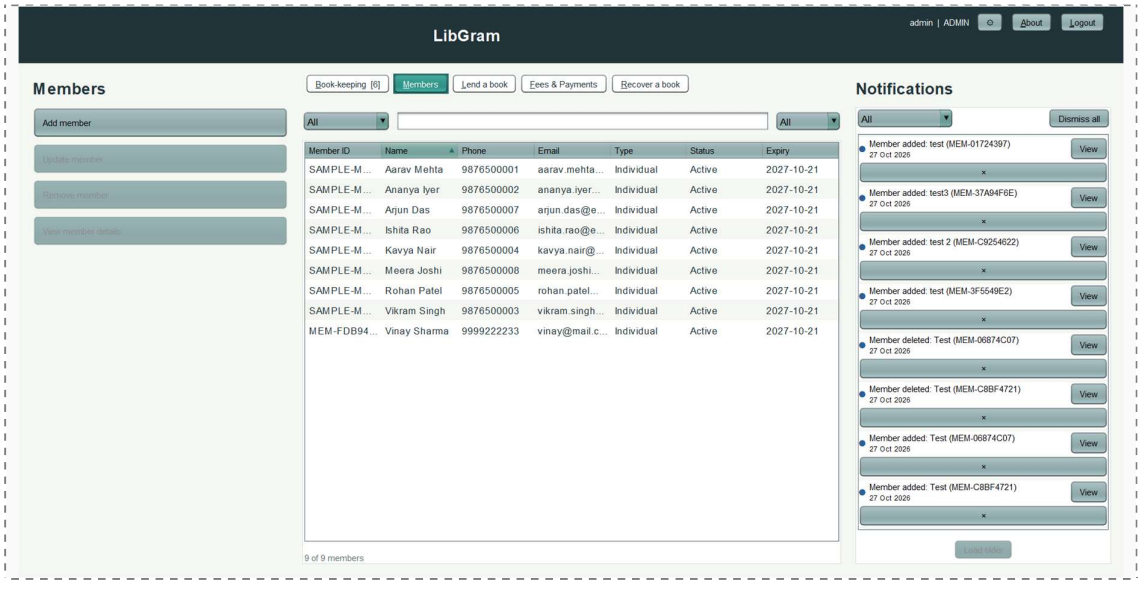


Figure 11: New lending screen with the lending list

18.2 Return Module

ReturnService records the return of one or more loans. The staff member selects the condition of each book: good, minor damage or severe damage. The service computes the charges

(Section 19.2), sets the return date and status, restores the available copy, writes the charges, and publishes a return event. When a reservation is waiting for that book, the return triggers promotion of the first eligible reservation to Ready.

A return date earlier than the issue date is rejected. A loan that is already closed cannot be returned again. The Recover a book menu groups the return, mark-lost, replace-member-card and overdue-list operations.

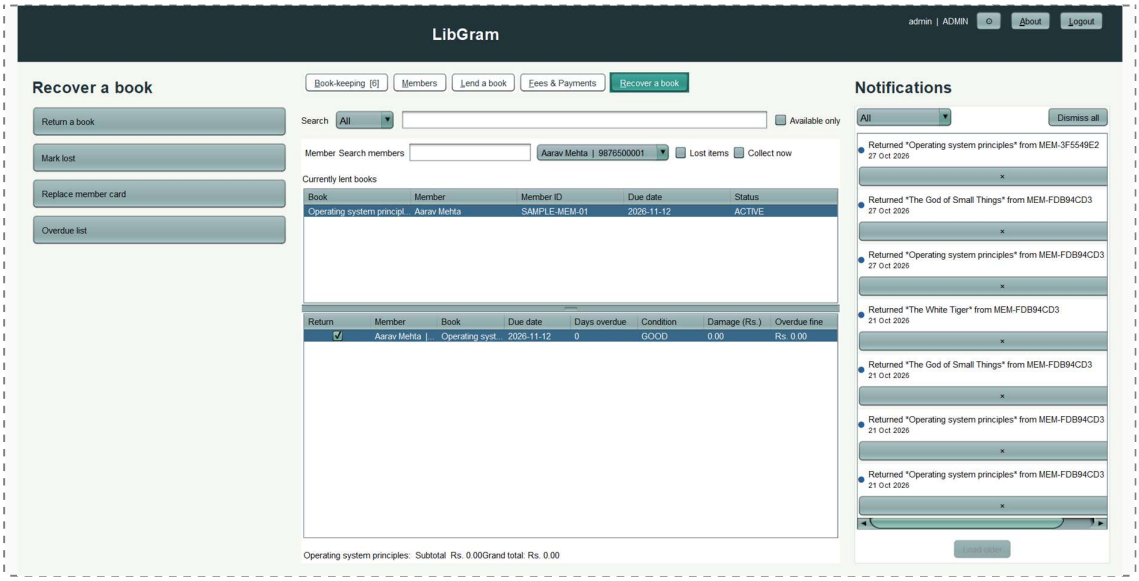


Figure 12: Return a book screen

18.3 Lost Books

LostBookService marks a loan lost either on request or automatically when the book has been overdue for longer than the lost-after period (default 60 days after the due date). The member is charged the replacement price of the book plus a lost-book processing fee (default Rs. 50.00) and any overdue fine accrued up to the date it was marked lost. Base and rent charges are not billed again. The date on which the book was marked lost is stored in the loan's existing return-date field, which keeps the CSV format unchanged. If a lost book is later returned within the refund window (default 180 days from the date it was marked lost), the return workflow recognises the loan as found and handles the refund; refund credits are settled by a refund payment and cannot be waived.

18.4 Reservations

ReservationService lets staff reserve a book that has no available copy. The service checks that the member is Active and unexpired, that no copy is currently lendable, that the member does not already hold the book or have a reservation for it, and that the member is under the reservation limit (default 3). Reservations have five states.

Table 33: Reservation states

State	Meaning
PENDING	Waiting for a copy to become available

State	Meaning
READY	A copy is being held for the member until the hold expires (default 3 days)
COLLECTED	The member borrowed the book
CANCELLED	Cancelled by staff or member
EXPIRED	The hold period passed without collection

When a copy is returned, the first pending reservation for that book is promoted to Ready. If a Ready reservation is cancelled or expires, the next pending reservation is promoted. The Reservations screen offers Ready, Pending and Closed filters and a search by member or book. An optional reservation fee (default Rs. 0.00) can be configured.

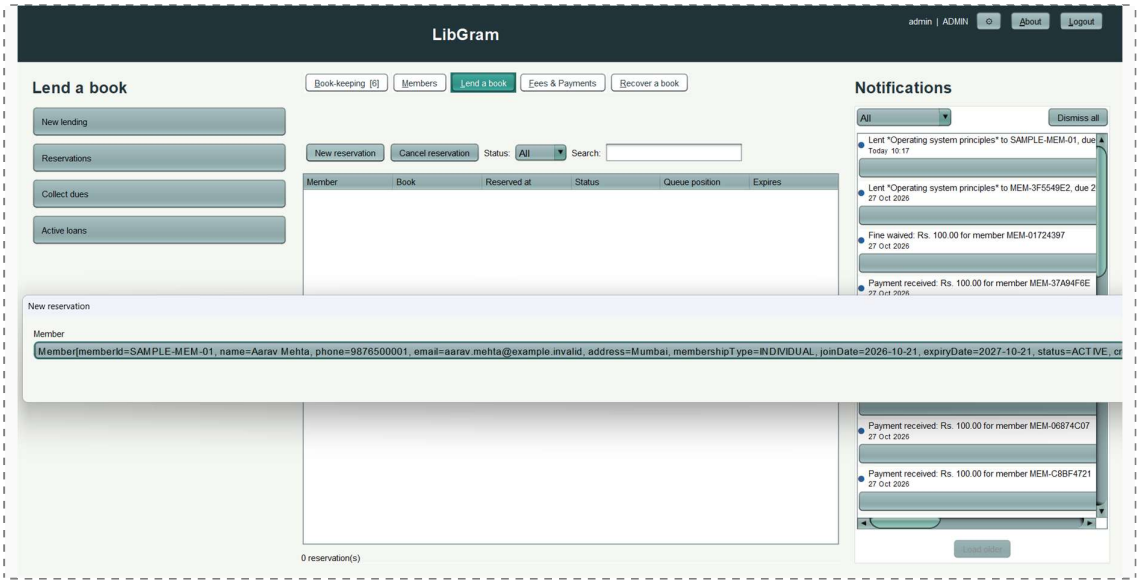


Figure 13: Reservations screen with filters

19. Implementation: Charges, Billing and Payments

19.1 Design of the Billing Rules

Billing is concentrated in ChargeCalculator, a final class of pure functions with no dependency on files or on the user interface. This makes the rules deterministic and easy to test. Every monetary value is a BigDecimal rounded to two decimal places using HALF_UP rounding, and negative charge lines are rejected. The calculator reads rent and penalty rates only from the pricing snapshot stored on the loan, so that later changes to settings do not alter the amount owed on a loan already issued.

19.2 Return Charges

Table 34: Charge lines produced when a book is returned

Line	Rule
BASE	The base charge recorded on the loan, if any
RENT	Rent amount multiplied by the number of rent units held; a unit is a day, a week (7 days) or a month (30 days) and at least one unit is billed
OVERDUE_FINE	Penalty per day multiplied by days overdue, limited to the per-item cap when the cap is greater than zero
DAMAGED_BOOK (minor)	The damage amount entered by staff
DAMAGED_BOOK (severe)	The replacement price of the book
PROCESSING_FEE	The damaged-book processing fee, added for severe damage
Zero lines	Lines with a zero amount are removed

19.3 Worked Examples

The following examples apply the rules with the default settings (penalty Rs. 2.00 per day, cap Rs. 100.00 per item). The figures illustrate the arithmetic of the rules. They are not recorded results from a particular library.

Table 35: Worked billing examples

Scenario	Calculation	Charge
Returned on or before the due date, good condition	No overdue days; no base or rent	Rs. 0.00 (no charge lines)
Due 15 Oct, returned 20 Oct, good condition	5 days overdue x Rs. 2.00	Rs. 10.00 overdue fine
Returned 60 days late	60 x Rs. 2.00 = Rs. 120.00, capped	Rs. 100.00 overdue fine
Rent Rs. 5.00 per day, held 19 days, on time	19 units x Rs. 5.00	Rs. 95.00 rent
Rent Rs. 20.00 per week, held 19 days	ceiling(19 / 7) = 3 units x Rs. 20.00	Rs. 60.00 rent

Scenario	Calculation	Charge
Severe damage, book price Rs. 450.00, processing Rs. 25.00	Price plus processing fee	Rs. 475.00 (two lines)
Lost book, price Rs. 300.00, 4 days overdue, fee Rs. 50.00	4 x Rs. 2.00 + Rs. 300.00 + Rs. 50.00	Rs. 358.00 (three lines)

19.4 Charge Lifecycle

A charge begins as UNPAID. A payment smaller than the balance makes it PARTIAL; a payment equal to the balance makes it PAID. An administrator may waive all or part of a charge, which records a waived amount and may make the charge WAIVED. The balance of a charge is its amount minus the waived amount minus the paid amount. Refund credits are charges of type REFUND and are settled through a refund payment rather than a normal payment.

19.5 Payments and Dues

PaymentService records payments against selected charges. A payment must be greater than zero and cannot exceed the total balance of the selected charges. The payment method is Cash, UPI, Card or Other. A receipt object summarises the allocation. The Collect dues and Payment history screens, DuesPanel and MemberDuesPanel, show outstanding amounts. The lending service refuses to lend to a member who has unpaid dues, which gives the dues a practical effect on circulation.

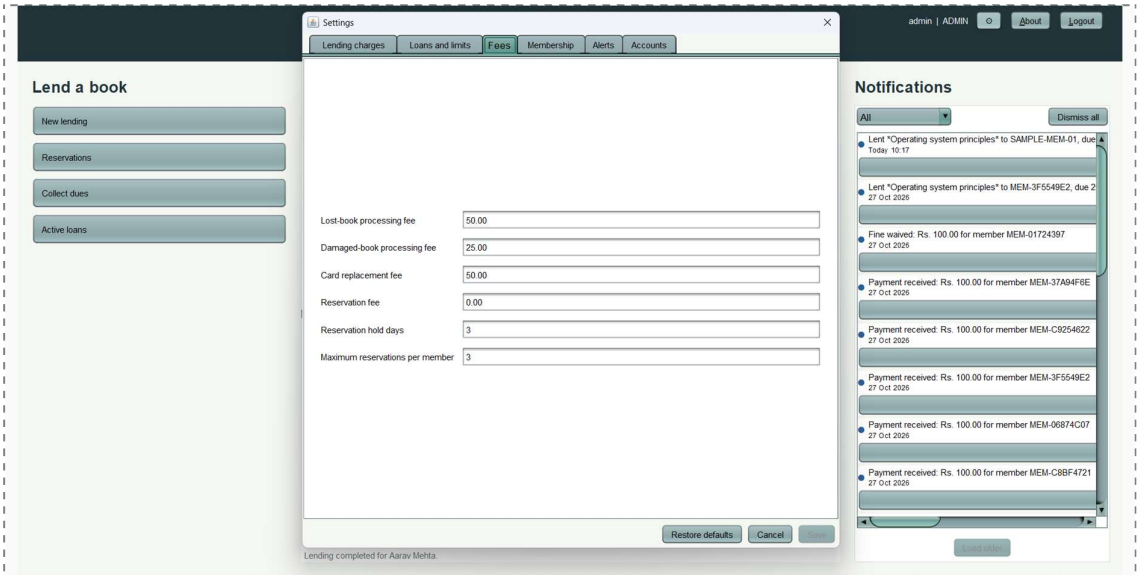


Figure 14: Collect payment screen and payment history

19.6 Fee Schedule Defaults

Table 36: Default charges and fees (SettingsDefaults)

Setting	Default	Notes
Base charge	Disabled, Rs. 0.00	Optional flat charge per loan

Setting	Default	Notes
Rent	Disabled, per day, Rs. 0.00	Day, week or month unit
Overdue penalty	Enabled, Rs. 2.00 per day	Cap Rs. 100.00 per item
Lost-book processing fee	Rs. 50.00	Added to the replacement price
Damaged-book processing fee	Rs. 25.00	Added for severe damage
Member-card replacement fee	Rs. 50.00	Charged when a card is replaced
Reservation fee	Rs. 0.00	Optional

20. Implementation: Notifications and Alerts

NotificationService stores notifications and NotificationPanel displays them. Notifications arrive from two sources: events (a book was lent, a payment was received) and conditions (a book is overdue, a membership is about to expire). Each has a type, a severity (INFO, WARNING or ALERT), the menu context it belongs to (BOOKKEEPING, MEMBERS, LENDING or RECOVERY), a reference to the related record and a deduplication key.

20.1 Notification Types

Table 37: Notification types

Group	Types
Books	BOOK_ADDED, BOOK_UPDATED, BOOK_REMOVED, LOW_STOCK, BOOK_OVERDUE
Members	MEMBER_ADDED, MEMBER_UPDATED, MEMBER_DEACTIVATED, MEMBER_DELETED, MEMBERSHIP_RENEWED, MEMBER_OVERDUE, MEMBERSHIP_EXPIRING, MEMBERSHIP_EXPIRED
Lending	BOOK_LENT, LENDING_BLOCKED, BOOK_RETURNED
Reservations	RESERVATION_CREATED, RESERVATION_READY, RESERVATION_EXPIRED, RESERVATION_COLLECTED
Billing	PAYMENT_RECEIVED, FINE_RAISED, FINE_WAIVED, REFUND_DUE, REFUND_PAID
Recovery	BOOK_LOST, BOOK_DAMAGED

20.2 The Alert Scanner

AlertScanner periodically examines the data and raises condition alerts: low stock when available copies are at or below the stock threshold, overdue books for each late loan, overdue members, memberships that expire within the configured number of days (default 14), and memberships that have already expired. The scanner uses the deduplication key so that the same condition does not produce a new notification on every scan. The scanner honours the alerts-enabled setting, while event notifications continue to be recorded.

20.3 Using Notifications

The notification panel shows the most recent items with paging, and an item can be dismissed. Each notification has a View action that switches to the related menu section and opens the referenced screen. Menu buttons show a count badge computed in a background worker so that the interface stays responsive. Old dismissed notifications are pruned at start-up after 365 days.

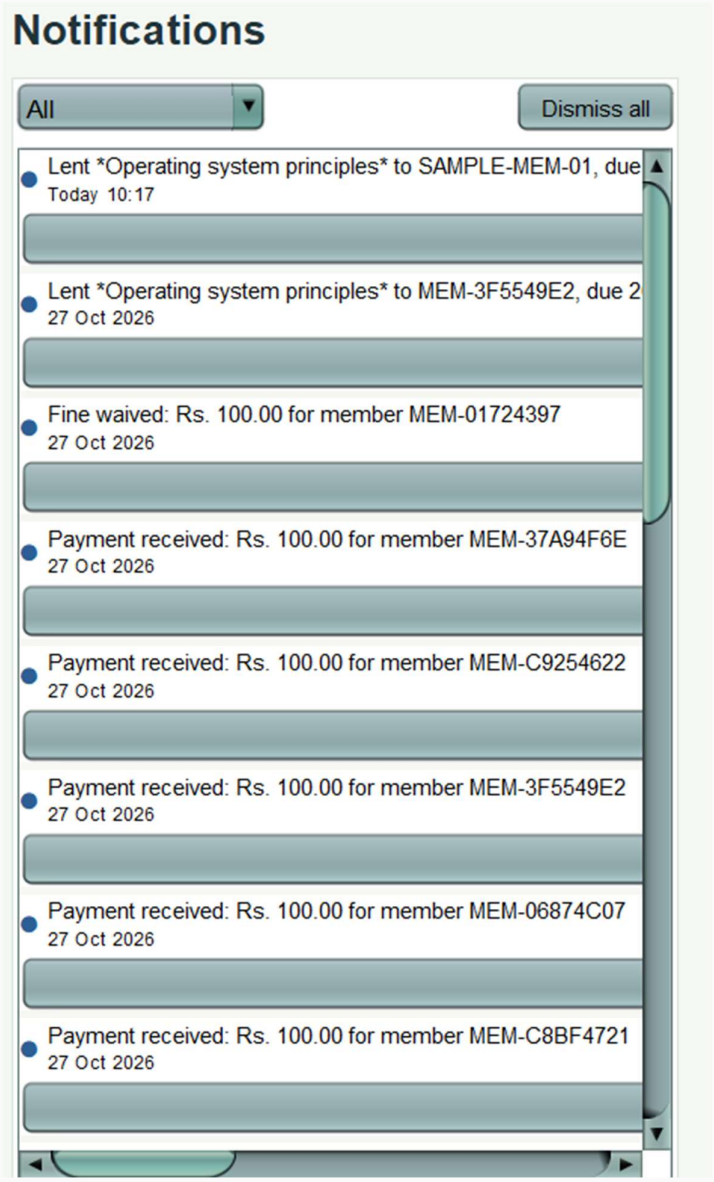


Figure 15: Notifications panel

21. Implementation: Authentication, Authorization and Settings

21.1 Authentication

On the first launch there is no account, so LoginPanel asks for an administrator account to be created. Later launches ask for a username and password. AuthService verifies the password by recomputing the hash, and AccountService creates and updates accounts.

Table 38: Password handling

Aspect	Implementation
Algorithm	PBKDF2WithHmacSHA256 from javax.crypto
Iterations	310,000
Salt	Random per user, generated with SecureRandom
Storage	Salt and hash only; the password itself is never stored
Minimum length	Eight characters, enforced wherever a password is set

A password is accepted only if its recomputed hash matches the stored one. Because each user has an individual salt, two users with the same password have different hashes.

21.2 Authorization

Permissions are decided by PermissionPolicy (Section 8.2) and enforced twice: by the interface, which hides or disables options, and by the services, which throw a SecurityException if an unauthorised or inactive user calls a method directly. A permission-matrix test checks this behaviour at the service level.

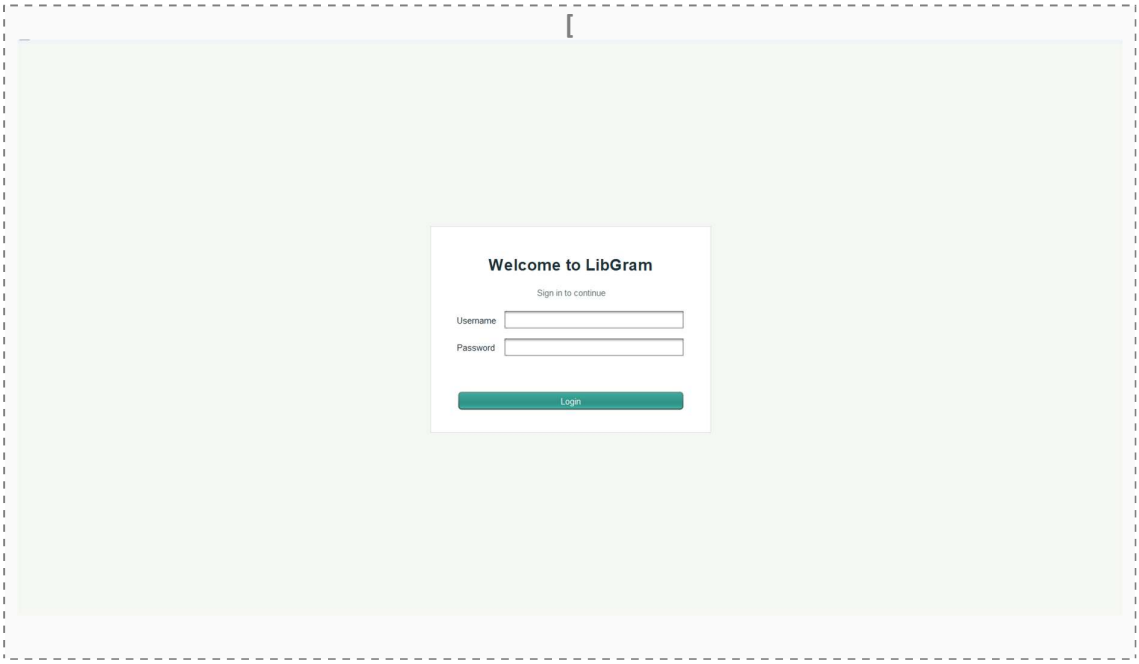


Figure 16: Login screen and first administrator setup

21.3 Settings and Configuration

SettingsService reads and writes settings, validates each value against a range, and publishes a change event so that open screens can refresh. SettingsDefaults is the single authoritative source of starting values; if a key is missing from the file, its default is used. Only administrators may open the settings dialog.

Table 39: Selected settings and defaults

Key	Default	Meaning
loan.defaultDays	14	Standard loan period in days
loan.maxDays	90	Longest allowed loan period
lost.afterDays	60	Days overdue after which a book is treated as lost
refund.windowDays	180	Period in which a refund can apply
reservation.holdDays	3	Days a Ready reservation is held
reservation.maxPerMember	3	Open reservations allowed per member
stock.lowThreshold	1	Available copies at or below which low stock is flagged
alerts.enabled	true	Switch for scanner alerts
alerts.membershipExpiryDays	14	Warning period before membership expiry
limit.INDIVIDUAL / FAMILY / SENIOR / TEMPORARY	3 / 6 / 3 / 2	Borrowing limit per membership type

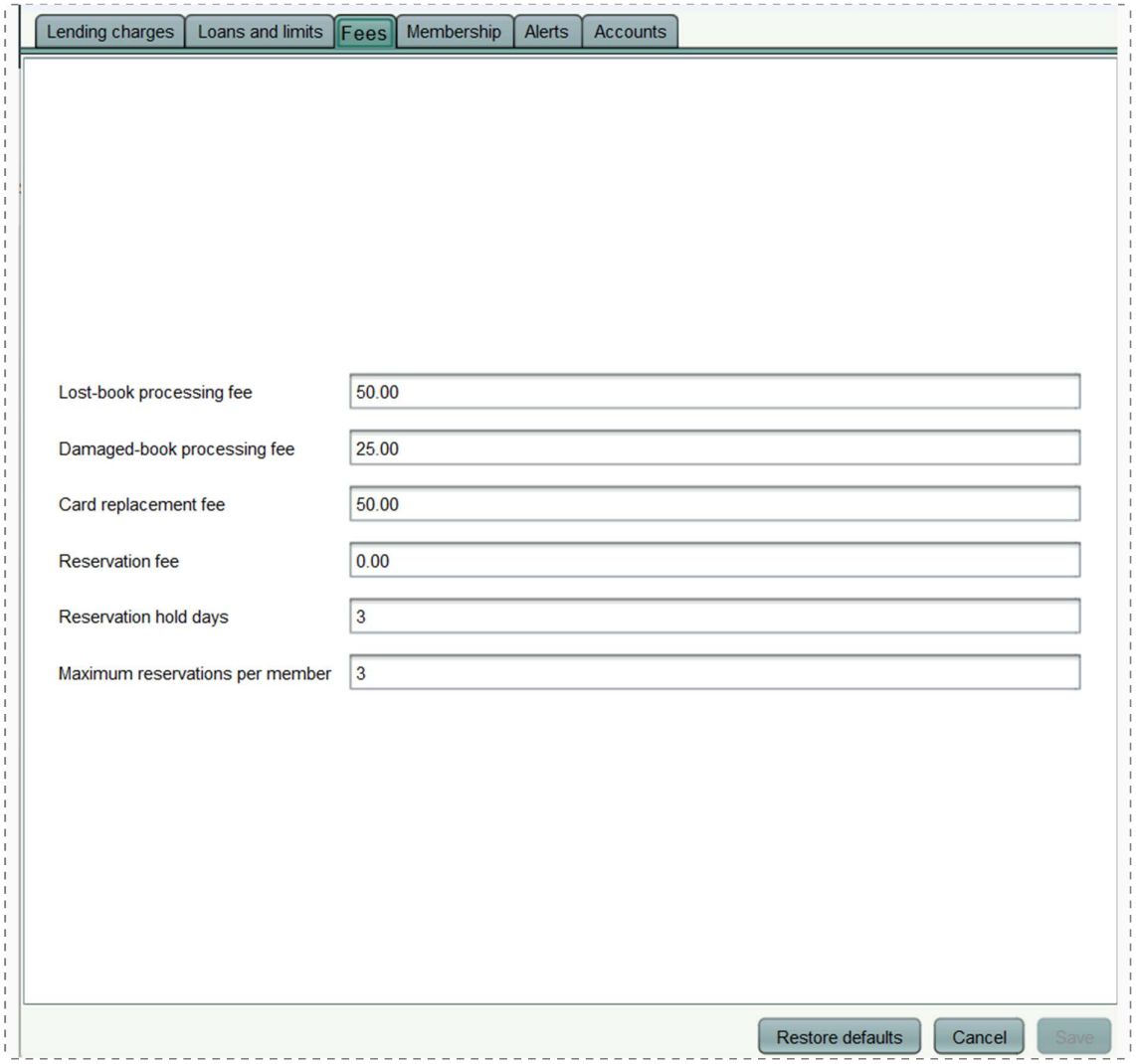


Figure 17: Settings dialog

22. Implementation: Persistence, Reliability and User Interface

22.1 CSV Persistence

Each entity type has a repository interface (for example BookRepository) and a CSV implementation (CsvBookRepository). The interfaces expose operations such as findAll and saveAll, so services work with lists of model objects and never with file paths. The shared helper CsvUtil handles quoting, so that commas, quotation marks and line breaks inside a field are stored and read back correctly. Files use UTF-8 text with a header row, and readers are defensive: a malformed row is detected rather than allowed to crash the application. Schema versions and migration code allow older files to remain readable.

22.2 Atomic Writes

A direct overwrite of a data file is dangerous: if the program or the computer stops half way, the file is left truncated. LibGram therefore writes new content to a temporary file in the same folder and then replaces the real file with a single atomic move. If the platform cannot perform an atomic move, it falls back to a normal replace. A disk error during the write leaves the previous file intact.

```
1. write new content to  books.txt.tmp
2. copy current books.txt to books.txt.bak           (recovery copy)
3. copy current books.txt to backups/books.txt-<date>.bak (daily snapshot, newest five
kept)
4. atomically move books.txt.tmp over books.txt
```

22.3 Backups and Recovery

Table 40: Backup and recovery behaviour

Situation	Behaviour
Every successful write	An adjacent .bak copy of the previous file is maintained
Daily	A dated copy is stored in backups/; the five newest per data file are kept and older ones are deleted
A data file cannot be parsed	The application first tries the adjacent .bak and reports the file that was recovered
The .bak also fails	The bad file is preserved with a .corrupt-<timestamp> suffix and the application starts with an empty dataset for that file; the damaged file is never overwritten
Manual restore	Close LibGram, keep a copy of the current file, copy the chosen backup over it using the original file name, restart and verify

> **Operational advice.** The backup folder lives on the same disk as the data. A library should periodically copy the whole storage folder to a pen drive or another computer so that a failed disk does not take both the data and the backups.

22.4 Single-Instance Lock

StorageLock opens a file named .libgram.lock in the storage folder and takes an exclusive operating-system file lock on it. A second process that tries to use the same folder fails to obtain the lock and is refused with a message. This prevents two running copies from overwriting each other's changes. The lock is released by a shutdown hook when the application exits. The lock protects one folder from two processes; it does not make the file store a multi-user database.

22.5 Logging and Error Handling

AppLog configures java.util.logging to write to storage/logs/libgram-YYYY-MM-DD.log. Log files older than 14 days are removed at start-up. Failures such as unreadable files, recovery actions and unexpected exceptions are logged with their causes. The design separates two audiences: the user sees a short, readable message, while the technical detail goes to the log. Passwords and password hashes should never appear in a log, and this practice is part of the project's coding guidance.

Table 41: Exception handling approach

Exception	Use
DuplicateIsbnException, DuplicateMemberEmailException, DuplicateMemberPhoneException	Subclasses of IllegalArgumentException that carry a precise validation message to the screen
IllegalArgumentException	Invalid input or a violated business rule; message shown to the user
IllegalStateException	An unexpected state, such as unreadable data, with the original exception as the cause
SecurityException	A permission check failed in a service
IOException	Storage failures propagated from the data layer

22.6 User Interface Design

The interface is built with standard Swing components under the Nimbus look and feel, installed by Theme. AppWindow, TitleBar and MainFrame form the shell. MenuSection defines the left-hand menu: Book-keeping, Members, Lend a book, Fees and Payments, and Recover a book, each with its own list of actions. A central card layout switches between the catalogue, member, dues and recovery panels. ProportionalLayout keeps the layout proportional when the window is resized.

Table 42: Menu structure of LibGram (MenuSection)

Menu	Actions
Book-keeping	Add book, Update book, Remove book, View details
Members	Add member, Update member, Remove member, View member details
Lend a book	New lending, Reservations, Collect dues, Active loans
Fees & Payments	Collect payment, Payment history
Recover a book	Return a book, Mark lost, Replace member card, Overdue list

- Long operations such as badge counting run in a SwingWorker so the interface stays responsive.
- The application starts on the Event Dispatch Thread, as Swing requires.
- Controls carry accessible names (for example "Search the current section") for assistive technologies.
- Options that a user is not permitted to use are disabled with an explanatory tooltip.
- A right-hand notification panel keeps alerts visible while staff work.

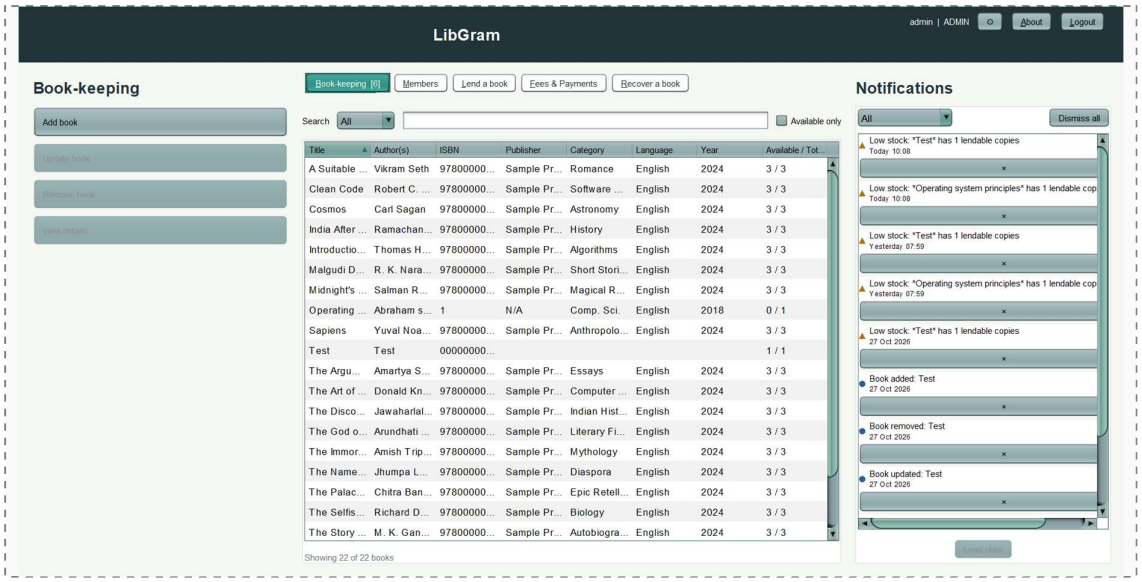


Figure 18: Main window of LibGram

23. LibGramPocket: The Personal Edition

23.1 Why a Second Application?

While studying the problem the team noticed that the same lending relationship occurs between individuals. A person lends a novel to a friend, or borrows a textbook from a classmate, and the record exists only in memory. LibGramPocket applies the same architecture to this smaller problem. It also serves as evidence that the layered design is reusable: model, services, data and ui packages are the same, with different entities and rules.

23.2 Features

- Sign up and sign in; each account has a private folder of CSV files, and account credentials are stored as salted PBKDF2 hashes.
- Owned-book catalogue with search, sorting and a remembered sort order.
- Contacts with name, phone, email and notes, and a detail view that shows books lent, books borrowed and amounts.
- Lending books to contacts and receiving them back; borrowing from contacts and returning to the lender.
- Optional pricing terms: a base charge, rent per day, week or month, and a daily penalty with a per-book cap; the terms in force are stored with the loan.
- Notifications and alerts for books due and amounts owed, with menu badges and a periodic scanner.
- Settings and remembered preferences, a change-password dialog and an About screen that shows the version, Java version and storage location.

23.3 Menu Structure

Table 43: Menu structure of LibGramPocket

Menu	Actions
Book-keeping	Add book, Update book, Remove book, View details
Lend a book	New lend, Receive a return, Active lends, Lend history, Contacts
Borrow a book	New borrow, Return to lender, Active borrows, Borrow history, Contacts

23.4 Data Model

Table 44: LibGramPocket entities

Entity	Fields
Book	Owned book with bibliographic details and copy counts; borrowed books appear with a Borrowed badge while active
Contact	contactId, name, phone, email, notes, createdAt, updatedAt, onlineUserId (reserved for future online features)

Entity	Fields
Loan	loanId, direction (lend or borrow), bookId, bookTitleSnapshot, contactId, issueDate, dueDate, returnDate, status, baseCharge, rentCharge, overduePenalty, damageCharge, returnCondition, paymentStatus, pricingSnapshot
PricingTerms	baseEnabled, baseAmount, rentEnabled, rentUnit, rentAmount, penaltyEnabled, penaltyPerDay, penaltyMaxPerBook
Bill and Bill.Line	Itemised charges with labels and exact two-decimal amounts; negative lines are rejected
Notification	Per-account alerts and events
User	Account with salted PBKDF2 hash

23.5 Account Isolation and Storage

Account data is stored under storage/users/<username>/, with books.txt, contacts.txt, loans.txt, notifications.txt and settings.txt. Credentials are in storage/users.txt. Usernames used for folders are restricted to 1 to 32 ASCII letters, digits, underscores or hyphens so that an account name cannot point outside its folder. On the first sign-in of each day the account files are copied to a backups folder and the newest five dated copies per file are kept. Atomic writes and the adjacent .bak copy work as in LibGram, and logs are written to storage/logs/libgrampocket-YYYY-MM-DD.log and kept for 14 days.

23.6 Behaviour Notes

- Lend and Borrow each remember the last confirmed pricing and otherwise use the defaults; new accounts start with charges disabled.
- A zero penalty cap means no cap; disabled pricing terms keep their remembered rates.
- A return action applies one return date to all selected loans.
- If more copies of an owned book are recorded as lent than exist, availability is repaired to zero when the application starts.
- Contact list previews are limited to 48 characters; full details remain in the contact view.

23.7 Differences from LibGram

Table 45: LibGram and LibGramPocket compared

Aspect	LibGram	LibGramPocket
Users	Shared library accounts with ADMIN and STAFF roles	One private account per person
Counterparty	Members with membership types, limits and expiry	Contacts with no membership
Direction of loans	Library lends to members	Lend to and borrow from contacts
Reservations and fees	Yes	Not included

Aspect	LibGram	LibGramPocket
Charges	Library fee schedule, lost and damaged book billing	Optional per-loan pricing terms
Roles	Permission policy	Not required (single owner)
Version	Phase 9 release	Version 1.0.0 (Phase 8)

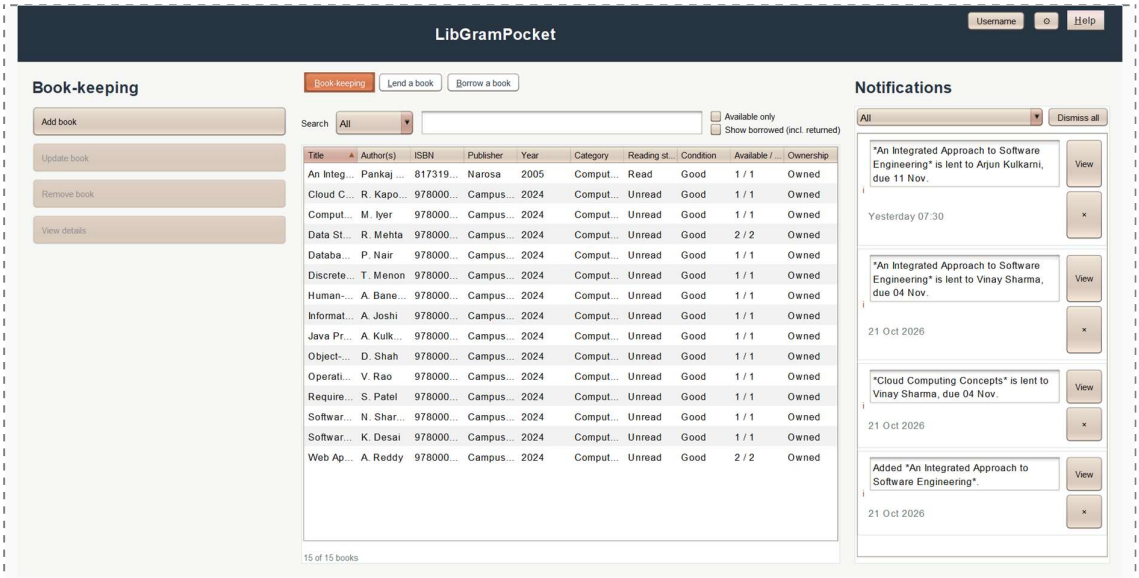


Figure 19: LibGramPocket main window and lend screen

The future plan for LibGramPocket is an online community feature in which contacts can be linked to other users and books can be exchanged within a wider group. The onlineUserId field on Contact is a placeholder for this. It is not implemented in version 1.0.0.

24. Object-Oriented Concepts Applied

The Object Oriented Technology Lab requires the demonstration of basic and intermediate features of Object-Oriented programming. This chapter maps each concept to concrete places in the code base, so that the evaluator can find them quickly.

24.1 Mapping of Concepts to the Project

Table 46: Object-Oriented features in LibGram

Concept	How it is used	Examples
Classes and objects	Each real-world entity and each responsibility is a class; objects are created and wired in Main	Book, Member, Loan, LendingService, CsvBookRepository
Encapsulation	State is held in private final fields; behaviour is exposed through methods; utility classes have private constructors; records validate in compact constructors	Charge.balance(), ChargeCalculator.Line, StorageLock
Abstraction	Services depend on interfaces that hide how data is stored	BookRepository, MemberRepository, LoanRepository, DuesProvider
Inheritance	Custom exceptions extend standard exception classes; UI classes extend Swing components	DuplicateIsbnException extends IllegalArgumentException
Polymorphism	The same interface reference can hold different implementations; listeners are called through their interface type	BookRepository -> CsvBookRepository; NoDuesProvider and RealDuesProvider
Interfaces	Contracts for repositories, event listeners and dues providers	SettingsChangedListener, LoanEventListener, ReturnEventListener
Enumerations	Fixed sets of values, some with fields and methods	MembershipType, ChargeStatus, MenuSection(label, options)
Records and immutability	Domain entities are immutable records; changes create a new object	Book, Member, Loan, Charge, Payment, Notification
Exception handling	Custom and standard exceptions separate validation, permission and storage failures	DuplicateMemberEmailException, SecurityException, IOException
Collections and generics	Typed lists and maps hold entities and settings	List<Book>, Map<String,String>, EnumMap<MenuSection,Long>
Packages	Four packages give high cohesion and one-way dependencies	model, data, services, ui
Nested and anonymous classes	Small helper types live inside the class that uses them	ChargeCalculator.Line, LendingPanel.CartEntry, table models
Static and final	Constants and stateless helpers are static; classes not meant for extension are final	LoanRules, SettingsDefaults, CsvUtil

Concept	How it is used	Examples
Lambdas and streams	Filtering and lookup of records	members.findAll().stream().filter(...) .findFirst()
Multithreading	Long work runs off the Event Dispatch Thread	SwingWorker for menu badge counts

24.2 Design Patterns Used

Table 47: Design patterns

Pattern	Where it appears	Benefit
Layered architecture	ui -> services -> data -> model	Separation of concerns
Repository	BookRepository and its CSV implementation	Storage can change without touching rules
Service layer	LendingService, ReturnService, PaymentService	Business rules in one place
Observer	Event listener interfaces and notification listeners	Loose coupling between actions and alerts
Null object	NoDuesProvider as a harmless DuesProvider	Avoids null checks in lending
Snapshot	Pricing and title snapshots stored with a loan	History is not rewritten by later edits
Constructor injection	Main builds repositories and passes them to services	Easy testing and clear dependencies

24.3 Why These Choices Matter

Object-Oriented design is not only a requirement of the course; it is what makes the application maintainable. Because the rules in ChargeCalculator know nothing about the screen, they can be tested by a plain Java program. Because services depend on repository interfaces, the CSV files could be replaced by a database later without changing the rules. Because notifications are produced by listeners, a new kind of alert can be added without changing the lending code.

25. Build, Run and Deployment

25.1 Requirements

Both applications require a Java Development Kit (JDK) 21 to build and a Java 21 runtime to run. The command java -version and the command javac -version should both report version 21 before building.

25.2 Building LibGram

```
Windows:      build.bat        then        run.bat
Linux/macOS:  ./build.sh        then        ./run.sh
```

The Windows build script compiles every source file with all compiler warnings enabled (-Xlint:all) into the out folder, runs the regression suite, stops if any test fails, and then packages the classes into LibGram.jar with Main as the entry point.

```
javac -Xlint:all -d out src\DemoData.java src\Main.java src\data\*.java
src\model\*.java src\services\*.java src\ui\*.java
call test\run-tests.bat
jar cfe LibGram.jar Main -C out .
```

25.3 Running

- **Normal launch:** double-click run.bat (Windows) or run ./run.sh, or run java -jar LibGram.jar.
- **First launch:** the application asks for an administrator account; the password must contain at least eight characters.
- **Development storage:** java -jar LibGram.jar --storage storage-demo selects an alternative data folder. This option is a development aid and is not shown in the interface.
- **Where data goes:** a storage folder is created beside the application; logs and backups are in subfolders.

25.4 Demo Data

A generator creates fictional data with clearly fake names such as Demo Member 1. The small dataset is intended for demonstrations; the large dataset is intended for performance checks.

Table 48: Demo data generators

Command	LibGram	LibGramPocket
java -cp out DemoData	Small fake dataset in storage-demo/	80 books, 20 contacts, 120 loans for account demo
java -cp out DemoData --large	10,000 books, 2,000 members, 20,000 loans, 30,000 charges	5,000 books, 500 contacts, 10,000 loans

25.5 Building LibGramPocket

LibGramPocket uses the same scheme: build.bat and run.bat on Windows, and sh build.sh and ./run.sh on Linux and macOS. Storage is resolved beside the compiled application or JAR, independent of the folder from which it is launched, and the --storage argument selects an alternative folder for development. To reset local data, close the application and remove its storage folder.

25.6 Deployment to a Library Computer

1. Confirm that a Java 21 runtime is installed on the library computer.
2. Copy LibGram.jar and the launch script to a folder on the computer, for example C:\LibGram.
3. Start the application and create the administrator account.
4. Open Settings and enter the library's loan period, fines, membership types and limits.
5. Enter the catalogue and member register, or add them gradually as books are issued.
6. Schedule a routine to copy the storage folder to a pen drive or another computer.

Where a library computer does not have a Java runtime, a self-contained runtime can be bundled with the application by using the Java tools jlink and jpackage. This would increase the size of the delivered package considerably, because the runtime would be shipped with it; the small size quoted for LibGram refers to the application JAR itself. Bundling is listed as future work in Chapter 28.

26. Testing

26.1 Testing Strategy

Testing combines automated regression tests with manual testing of the user interface. The automated tests are written as plain Java programs with a main method, which avoids any external testing framework and keeps the project dependency-free. A runner script compiles the sources with all compiler warnings enabled, executes each test, prints a PASS or FAIL line for each and returns a non-zero exit code if any test fails. The build script runs this suite before it creates the JAR, so a failing test prevents a release.

Table 49: Levels of testing

Level	Purpose	Examples
Unit	Verify a rule in isolation	ChargeCalculatorTest, CsvUtilTest
Service	Verify business workflows against temporary storage	LendingServiceTest, MemberServiceTest, CatalogServiceTest, ReservationServiceTest, AccountServiceTest
Workflow	Verify multi-step scenarios end to end	PhaseSixWorkflowTest, PhaseSixBillingTest, PhaseEightNotificationTest
Robustness	Verify recovery, migration, locking and consistency	RepositoryRecoveryTest, MigrationTest, InvariantSequenceTest, PhaseNineHardeningTest
Security	Verify the permission matrix	PermissionMatrixTest
Performance smoke	Verify behaviour with large data sizes	PerformanceSmokeTest
Manual	Verify screens, navigation and usability	Walkthrough of each menu with demo data

26.2 Test Inventory of LibGram

The runner script executes the following tests. Test data is created in temporary storage and uses fake records, so a test never touches real library data.

Table 50: LibGram regression tests

Test	Area verified
AccountServiceTest	Account creation, password rules and hashing
CatalogServiceTest	Book add, update, remove, search and duplicate ISBN
LendingServiceTest	Issuing books and validation failures
MemberServiceTest	Member creation, required fields, duplicate email and phone, search and status filtering
ReservationServiceTest	Reservation creation, eligibility checks and status changes

Test	Area verified
ChargeCalculatorTest	Charge boundaries: overdue days, cap, rent units, damage and lost-book lines
CsvUtilTest	Quoting, commas, quotation marks and line breaks in fields
RepositoryRecoveryTest	Fallback to backup copy and preservation of corrupt files
PermissionMatrixTest	Role permissions at the service level
MigrationTest	Older file formats remain readable
InvariantSequenceTest	Sequences of operations do not leave loans, charges or payments inconsistent
PhaseOneSmokeTest	Application start-up and layout proportions
PhaseSixBillingTest	Billing rules, including the disabled-rent pricing snapshot case
PhaseSixWorkflowTest	Detailed lending, return, charge and payment workflow and charge states
PhaseEightNotificationTest	Notification paging, dismissal, scanner deduplication and honouring of settings
PhaseNineHardeningTest	Hardening features: backups, recovery and locking
PerformanceSmokeTest	Operation with 10,000 books, 2,000 members, 20,000 loans and 30,000 charges
StoragePathsTest	Resolution of the storage folder

26.3 Test Inventory of LibGramPocket

Table 51: LibGramPocket regression tests

Test	Area verified
AuthServiceTest	Sign-up, sign-in and password handling
BookServiceTest	Owned-book operations and search
ContactServiceTest	Contact management and duplicate detection
LendingServiceTest	Lend, borrow and return workflows
ChargeCalculatorTest, PricingTermsTest	Pricing terms and charge calculation
NotificationServiceTest, AlertScannerTest	Notifications and due or owed alerts
SettingsServiceTest, CsvSettingsRepositoryTest	Settings storage and defaults
CsvRoundTripTest, CsvLoanRepositoryTest, CsvMigrationTest	CSV reading, writing and migration
CsvRecoveryTest, StorageSafetyTest, StorageLockProbe	Recovery, safe storage and the single-instance lock
StoragePathsTest, AppLogTest	Storage folders and logging
PhaseEightInvariantTest	Consistency of loans and inventory

Test	Area verified
DisplayFormatTest	Formatting of values for display
MainFrameInitializationTest, BorrowBookPanelLayoutTest, PricingFormInitializationTest, LendBookPickerPanelTest	Screen initialisation and layout
DemoDataTest, PerformanceCheck	Demo data generation and large-dataset behaviour

26.4 Designed Test Cases

The tables below describe representative test cases by scenario and expected result. They follow the rules implemented in the services. Each corresponds to behaviour exercised by the regression suite or by the manual walkthrough.

Table 52: Test cases: authentication and members

ID	Scenario	Action	Expected result
TC-01	First launch with no account	Start application	Administrator setup is shown
TC-02	Correct credentials	Sign in	Main window opens
TC-03	Wrong password	Sign in	Sign-in refused with a message
TC-04	Short password	Create account with fewer than eight characters	Rejected
TC-05	Staff user opens settings	Select settings	Not permitted
TC-06	Add member with a unique phone and email	Save member	Member is created
TC-07	Add member with an existing email	Save member	DuplicateMemberEmailException message shown
TC-08	Add member with an existing phone	Save member	DuplicateMemberPhoneException message shown
TC-09	Staff user edits a member	Update member	Not permitted; view only

Table 53: Test cases: books and circulation

ID	Scenario	Action	Expected result
TC-10	Add a book with a new ISBN	Save book	Book appears in the catalogue
TC-11	Add a book with an existing ISBN	Save book	DuplicateIsbnException message shown
TC-12	Search by author	Enter text and choose Author	Only matching books are listed
TC-13	Lend an available book to an active member	Lend	Loan created; available copies reduced by one
TC-14	Lend a book with no available copy	Lend	Refused

ID	Scenario	Action	Expected result
TC-15	Lend to a suspended member	Lend	Refused: member is not Active
TC-16	Lend to a member with expired membership	Lend	Refused: membership expired
TC-17	Lend beyond the borrowing limit	Lend	Refused: limit exceeded
TC-18	Lend to a member with unpaid dues	Lend	Refused: unpaid dues
TC-19	Lend with several failing checks	Lend	All failures are reported together and nothing is saved
TC-20	Return a book in good condition on time	Return	Copy restored; no charge
TC-21	Return the same loan twice	Return	Refused: loan already closed

Table 54: Test cases: reservations, charges and payments

ID	Scenario	Action	Expected result
TC-22	Reserve a book that has available copies	Reserve	Refused: lend it instead
TC-23	Reserve an unavailable book	Reserve	Reservation created as Pending
TC-24	Reserve the same book twice	Reserve	Refused: reservation already exists
TC-25	Exceed the reservation limit	Reserve	Refused: limit reached
TC-26	A copy is returned for a reserved book	Return	First pending reservation becomes Ready
TC-27	Return 5 days late with default penalty	Return	Overdue fine of Rs. 10.00
TC-28	Return 60 days late	Return	Fine capped at Rs. 100.00
TC-29	Return a book with severe damage	Return	Replacement price plus processing fee charged
TC-30	Pay more than the balance	Payment	Refused: payment exceeds balance
TC-31	Pay a part of a charge	Payment	Charge becomes Partial
TC-32	Pay the full balance	Payment	Charge becomes Paid

Table 55: Test cases: reliability and security

ID	Scenario	Action	Expected result
TC-33	Unreadable data file with a good .bak	Start application	Recovered from .bak and reported

ID	Scenario	Action	Expected result
TC-34	Unreadable data file and unreadable .bak	Start application	Bad file kept as .corrupt-<timestamp>; empty dataset for that file
TC-35	Second instance on the same storage folder	Start second copy	Refused by the storage lock
TC-36	Write interrupted	Simulated failure during save	Previous file intact
TC-37	Field containing commas, quotes and line breaks	Save and reload	Value is identical after reload
TC-38	Older file format	Start application	File is read through migration
TC-39	Large dataset	Run performance smoke test	Operations complete without failure
TC-40	Service called by an unauthorised user	Call service directly	SecurityException

26.5 Test Execution and Results

The team built the final version of both applications and ran the regression suites as part of the final verification before submission. The test runner reports a PASS or FAIL line for each test and an overall result. The output of the final run is attached to this report as evidence in the figure below. The capacity figures reported in this document (5,000 books and 500 members for LibGram, and 5,000 books for LibGramPocket) are those verified by the team in its own testing.

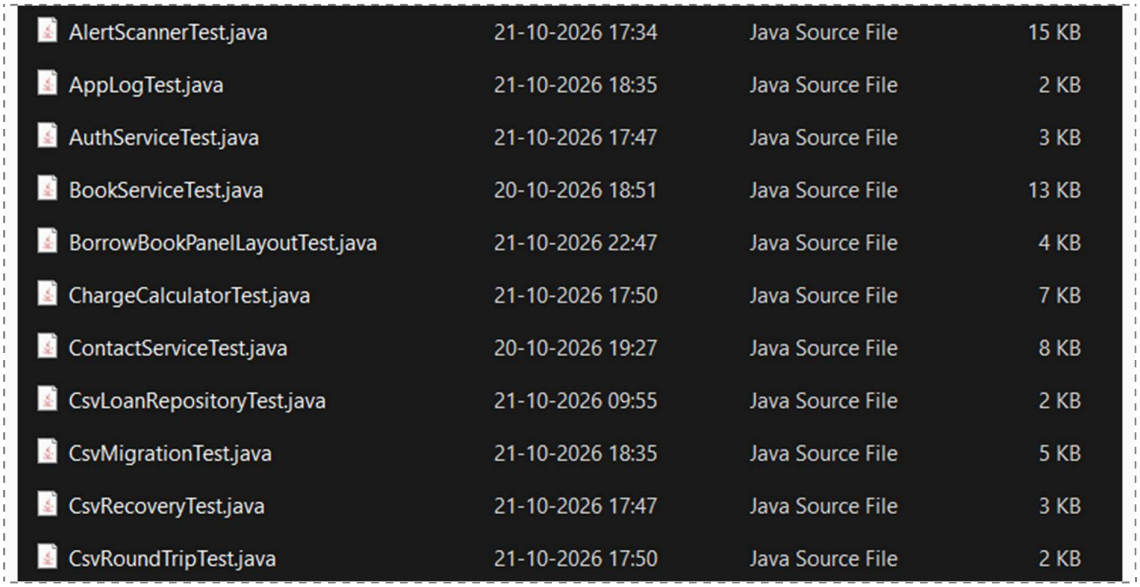


Figure 20: Successful build and regression test output

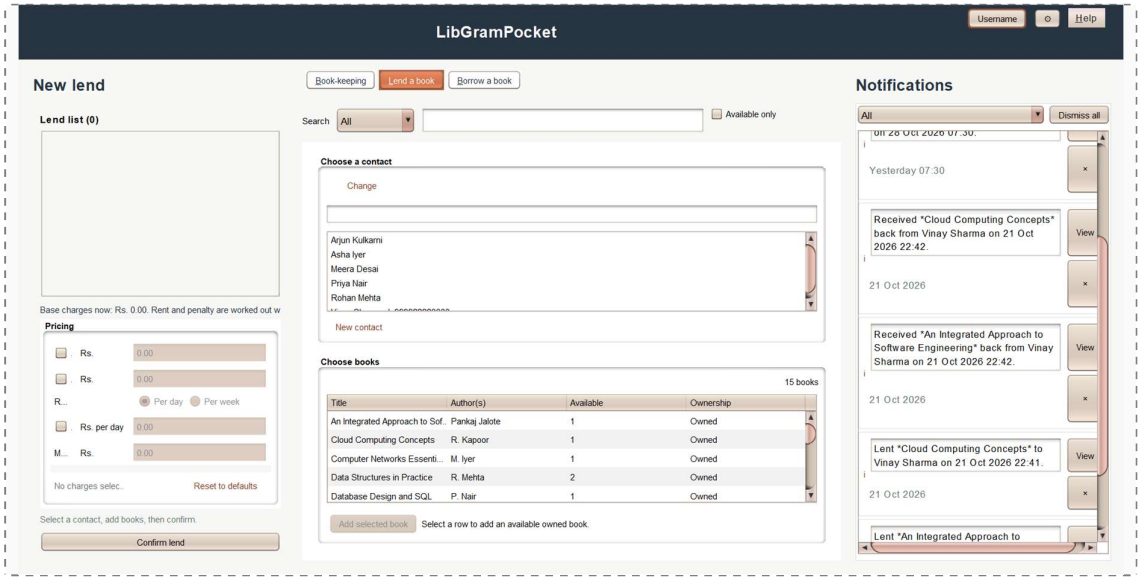


Figure 21: Regression test output for LibGramPocket

Interpretation. A passing suite shows that the rules and workflows covered by the tests behave as specified. It does not by itself prove the absence of defects, the quality of every screen, or performance on every computer.

27. Results, Limitations and Future Enhancements

27.1 Results Against Objectives

Table 56: Objectives and outcome

Objective	Outcome
Catalogue and member management with validation	Implemented with duplicate checks and search
Automated lending, returns and reservations with eligibility checks	Implemented; every check is reported together
Consistent charge calculation	Implemented with BigDecimal and a pricing snapshot
Payments and dues	Implemented with Partial, Paid and Waived states
Notifications	Implemented with events, scanner alerts and deduplication
Authentication and permissions	Implemented with PBKDF2 hashes and a permission policy
Data safety	Implemented: atomic writes, backups, recovery, lock and logs
Small footprint	LibGram JAR about 376 KB; LibGramPocket JAR about 276 KB
Personal edition	LibGramPocket implemented to version 1.0.0
Object-Oriented features	Demonstrated across both code bases (Chapter 24)

27.2 Limitations

- **Single machine:** the application uses local files and a single-instance lock. It is not a multi-user, networked or multi-branch system.
- **Java runtime:** a Java 21 runtime must be present on the computer. The small JAR size does not include the runtime.
- **CSV storage:** CSV is simple and transparent but is less suitable for complex queries, large reports and strict transactions across many files.
- **Capacity:** field-tested for up to 5,000 books and 500 members. Larger collections may work but have not been verified in this way.
- **Field inquiry:** the inquiry covered two to three rural libraries and is not a statistical survey.
- **No barcode or card-reader integration:** records are entered through the keyboard.
- **Local language:** the interface is in English.
- **Backups on the same disk:** library staff must copy the storage folder elsewhere for protection against disk failure.

27.3 Future Enhancements

Table 57: Future work

Enhancement	Description
Bundled runtime and installer	Package the application with a private Java runtime using jlink and jpackage, and add an installer with desktop shortcuts, so that no separate Java installation is needed
Export and import wizard	An administrator tool to move data between computers and to import an existing catalogue from a spreadsheet
Online community for LibGramPocket	Link contacts to other users and create an online marketplace where people can exchange books with a wider community
Lost-book workflow for LibGramPocket	A workflow for books that are not returned
Barcode support	Scan ISBNs and member cards to speed up lending
Richer reports	Circulation statistics, most-borrowed books and dues summaries
Audit trail	A tamper-evident record of who changed what
Local-language interface	Marathi and Konkani labels for staff comfort
Database-backed option	A switch to an embedded database for libraries that outgrow CSV, made easier by the repository interfaces
Accessibility review	Keyboard navigation, contrast and screen-reader checks across all screens

These items are recommendations. They are not implemented features of the current release.

27.4 Privacy, Security and Ethics

Library records are personal data. A library using LibGram should follow a few simple practices.

- Keep the storage folder on a computer that only authorised staff can use.
- Use a strong administrator password and do not share accounts between staff.
- Do not publish or share real data; use demo data for demonstrations and screenshots.
- Keep backups in a safe place and delete old copies that are no longer needed.
- Collect only the member details that the library actually needs.

The project itself used fictional data for all demonstrations and tests, and the application does not transmit data over a network.

27.5 Risk Analysis

Table 58: Risks and mitigations

Risk	Where it matters	Mitigation in the project
Accidental deletion of records	Catalogue and members	Role-based permissions; daily backups; notifications for removals

Risk	Where it matters	Mitigation in the project
Malformed or damaged CSV data	All data files	Defensive parsing; automatic recovery from .bak; corrupt files preserved
Disk or power failure during a write	All data files	Atomic replacement; previous file remains intact
Lost credentials	Authentication	Administrator can manage accounts; data files remain readable text
Wrong charge calculation	Billing	Pure calculator; BigDecimal; pricing snapshot; boundary tests
Two processes using the same data	Storage	Exclusive file lock on the storage folder
Library without Java	Deployment	Install Java 21, or bundle a runtime in future work
Disk failure destroying data and backups	Storage	Advice to copy the storage folder to a second device

27.6 Project Management

The project was carried out by a team of two students under the guidance of a faculty member. Work followed the iterative phases described in Chapter 13: field inquiry and comparison, brainstorming and design, implementation module by module, testing, hardening, documentation and the demonstration. Version control was kept in a Git repository on GitHub.

Table 59: Project activities

Activity	Output
Field inquiry	Understanding of how rural libraries keep records
Competitor comparison	Table of existing systems and their weaknesses
Brainstorming and decisions	Idea map; accepted and rejected ideas
Requirements and design	Use-case, activity, sequence, data-flow and class diagrams
Implementation of LibGram	Application in Java 21 and Swing
Implementation of LibGramPocket	Personal edition on the same architecture
Testing and hardening	Regression suites; recovery, backup, lock and logging
Documentation	README files and this project report
Demonstration	Working product demonstration and presentation

28. Conclusion

This project began with a small, concrete injustice: a return that was made but not recorded, and a member who was told otherwise. It grew into an application that answers the question that incident raised, which is how a small library can keep records that are trustworthy, searchable and safe, without paying for software it cannot run or staff it does not have.

LibGram provides the catalogue, membership, lending, return, reservation, billing, payment, notification, security and settings features that a small library needs. It does so in a package of a few hundred kilobytes, stores its data in readable files, and defends that data with atomic writes, rolling backups, recovery, a single-instance lock and logs. LibGramPocket extends the same design to individuals who lend and borrow books.

From an academic point of view, the project shows classes and objects, encapsulation, abstraction, inheritance, polymorphism, interfaces, enumerations, records, exceptions, collections, nested classes, lambdas and multithreading working together in a layered architecture with repository, service and observer patterns. A plain-JDK regression suite checks the main rules and workflows.

The system has limits, and this report has stated them: it is a single-computer system, it needs a Java runtime, and it was tested at the scale of a small library. The most useful next steps are a bundled runtime with an installer, an export and import tool, and the online community features planned for LibGramPocket. For the libraries that motivated the project, however, the current release already offers something that a paper register cannot: a record that does not forget.

29. References

1. Oracle Corporation. Java Platform, Standard Edition 21 API Specification. docs.oracle.com.
2. Oracle Corporation. The Java Tutorials: Creating a GUI with Swing. docs.oracle.com/javase/tutorial.
3. Oracle Corporation. Java Platform, Standard Edition Tools Reference: jar, jlink and jpackage. docs.oracle.com.
4. Moriarty, K., Kaliski, B., Rusch, A. PKCS #5: Password-Based Cryptography Specification Version 2.1, RFC 8018, IETF, 2017.
5. Gamma, E., Helm, R., Johnson, R., Vlissides, J. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.
6. Fowler, M. Patterns of Enterprise Application Architecture. Addison-Wesley, 2002.
7. Bloch, J. Effective Java, 3rd Edition. Addison-Wesley, 2018.
8. Object Management Group. Unified Modeling Language (UML) Specification.
9. Evergreen ILS Project. Evergreen Open Source Library Software: project documentation.
10. SLiMS Project. Senayan Library Management System: project documentation.
11. eGranthalaya: Library automation software project documentation.
12. Project source repository: github.com/NGodwill10/MiniProject (LibGram and LibGramPocket README files, source code and tests).

> **Note.** Replace or extend this list with the exact pages and course materials that the team consulted, in the citation style required by the institution.

Appendix A. Glossary

Table 60: Glossary of terms

Term	Meaning
ILS	Integrated Library System, software that manages library catalogue and circulation
OPAC	Online Public Access Catalogue, a catalogue search interface for readers
CSV	Comma-separated values, a plain-text table format
Swing	Java's standard toolkit for desktop graphical user interfaces
JAR	Java Archive, a packaged Java application
PBKDF2	Password-Based Key Derivation Function 2, used to hash passwords with a salt and many iterations
Salt	Random data added to a password before hashing so equal passwords give different hashes
Atomic write	A write that either fully succeeds or leaves the old file unchanged
Repository	An object that hides how a kind of data is stored
Observer pattern	A design in which listeners are told when something happens
Snapshot	A copy of values (such as pricing) saved with a record so later changes do not alter it
Dues	Amounts a member owes the library
EDT	Event Dispatch Thread, the thread on which Swing screens must be updated

Appendix B. Demonstration Plan

A short demonstration of five minutes can use the small demo dataset and follow the script below.

Table 61: Demonstration script

Step	Action	Expected result	Feature shown
1	Start the application and sign in	Main window opens	Authentication
2	Open Book-keeping and search for a title	Matching books are listed	Catalogue and search
3	Open Members and view a member	Detail view shows borrowed books and dues	Member management
4	Lend two books to a member	Loans created, availability reduced	Lending
5	Try to lend to a suspended member	Refused with a clear message	Validation
6	Reserve an unavailable book	Reservation shown as Pending	Reservations
7	Return a book 5 days late	Overdue fine of Rs. 10.00 at default settings	Billing
8	Collect a payment	Charge becomes Paid	Payments
9	Open the notification panel	Events listed with View action	Notifications
10	Show the storage folder with backups and logs	Dated backups and log file visible	Reliability

Appendix C. Command Reference

Table 62: Commands

Command	Purpose
build.bat / ./build.sh	Compile, run tests and package the JAR
run.bat / ./run.sh	Start the application
test\run-tests.bat / ./test/run-tests.sh	Run the regression suite
java -jar LibGram.jar --storage storage-demo	Start with a development storage folder
java -cp out DemoData	Create small demo data
java -cp out DemoData --large	Create the large performance dataset
java -cp out PhaseSixWorkflowTest	Run the detailed billing workflow test directly

Appendix D. Final Submission Checklist

- Student details, course, affiliation and guide are complete on the cover and certificate.
- The certificate and declaration use the wording required by the institution.
- All diagrams carry figure numbers and captions.
- Screenshots replace every screenshot placeholder in the report.
- Test output screenshots are inserted in Chapter 26.
- The table of contents is updated in Word after final layout (right-click, Update Field).
- References follow the citation style required by the institution.